

Performance Analysis

[Reattempt](#)[Download Certificate](#)

Detailed analysis of your performance in the AWS Certified Solutions Architect Associate - SAA C03 exam.

Answer Distribution

0

Correct

0

Incorrect

65

Unattempted

Domain Performance

Application Integration	0%
Compute	0%
Security	0%
Storage	0%
Networking	0%
Databases	0%
Serverless	0%
Monitoring and Logging	0%

Final Score

**0/65**

Time Taken

**130m 0s**

Total time: 130 minutes

Question Analysis

✓ Correct	0
✗ Incorrect	0
⊖ Unattempted	65
🔖 Reviewed	0

Question-wise Analysis

Expand All

Collapse All

Question 1

Not Attempted

Application Integration



A solutions architect is designing an asynchronous application to process credit card data validation requests for a bank. The application must be secure and be able to process each request at least once. Which solution will meet these requirements MOST cost-effectively?

- ✓ Use AWS Lambda event source mapping. Set Amazon Simple Queue Service (Amazon SQS) standard queues as the event source. Use AWS Key Management Service (SSE-KMS) for encryption. Add the kms:Decrypt permission for the Lambda execution role.

This option uses standard SQS queues, which provide at-least-once delivery, and SSE-KMS encryption for security. The Lambda execution role needs decrypt permissions to access the encrypted data. This is a cost-effective solution as it avoids the higher cost of FIFO queues.

- ⊗ Use AWS Lambda event source mapping. Use Amazon Simple Queue Service (Amazon SQS) FIFO queues as the event source. Use SQS managed encryption keys (SSE-SQS) for encryption. Add the encryption key invocation permission for the Lambda function.
FIFO queues guarantee exactly-once processing, which is not a requirement here, and are more expensive than standard queues. SQS managed encryption keys (SSE-SQS) do not require adding encryption key invocation permission to the Lambda function.
- ⊗ Use the AWS Lambda event source mapping. Set Amazon Simple Queue Service (Amazon SQS) FIFO queues as the event source. Use AWS KMS keys (SSE-KMS). Add the kms:Decrypt permission for the Lambda execution role.
FIFO queues guarantee exactly-once processing which is an overkill and more expensive. The application only requires at-least-once processing. While SSE-KMS provides encryption, FIFO queues are not cost-effective for this scenario.
- ⊗ Use the AWS Lambda event source mapping. Set Amazon Simple Queue Service (Amazon SQS) standard queues as the event source. Use AWS KMS keys (SSE-KMS) for encryption. Add the encryption key invocation permission for the Lambda function.
While standard queues meet the 'at least once' requirement, adding encryption key invocation permission to the Lambda *function* is incorrect. Permissions are applied to the Lambda *execution role*.

Overall Explanation:

The most cost-effective solution is to use standard SQS queues with AWS Lambda event source mapping and SSE-KMS encryption. Standard queues provide at-least-once delivery, fulfilling the application's requirement. SSE-KMS ensures data security, and the Lambda execution role requires kms:Decrypt permission. FIFO queues guarantee exactly-once delivery which is overkill and would incur additional cost. Also, permissions are attached to Lambda execution roles, not the functions themselves.

[View Discussions](#)[Ask AI](#)

Question 2

Not Attempted

[Compute](#)

A company uses multiple AWS accounts for development. Developers often provision oversized Amazon EC2 instances, leading to budget overruns. The company needs a centralized solution to restrict resource creation in these development accounts with minimal development effort.

- ⊗ Develop AWS Systems Manager templates that use an approved EC2 creation process. Use the approved Systems Manager templates to provision EC2 instances.

Developing Systems Manager templates requires significant development effort to define and maintain the approved EC2 creation processes.

- ✔ Use AWS Organizations to organize the accounts into organizational units (OUs). Define and attach a service control policy (SCP) to control the usage of EC2 instance types.

AWS Organizations and SCPs provide a centralized and declarative way to manage permissions across multiple accounts with minimal development effort. You can define the allowed EC2 instance types in the SCP and apply it to the development OUs.

- ⊗ Configure an Amazon EventBridge rule that invokes an AWS Lambda function when an EC2 instance is created. Stop disallowed EC2 instance types.

This approach requires development effort to create and maintain the EventBridge rule and Lambda function. It also involves reacting to events after the resource has been created, which is less proactive than SCPs.

- ⊗ Set up AWS Service Catalog products for the staff to create the allowed EC2 instance types. Ensure that staff can deploy EC2 instances only by using the Service Catalog products.

Creating and maintaining Service Catalog products requires development effort to define the allowed EC2 instance types and associated configurations.

Overall Explanation:

AWS Organizations with Service Control Policies (SCPs) is the best solution because it allows for centralized management and restriction of EC2 instance types across multiple AWS accounts with the least development effort. SCPs are declarative and easy to manage, making it the most efficient option. The other options require more development and maintenance overhead.

 View Discussions

 Ask AI

A company wants to use artificial intelligence (AI) to determine the quality of its customer service calls. The company currently manages calls in four different languages, including English, and plans to add new languages in the future. The company lacks the resources for regular maintenance of machine learning (ML) models. The company needs to create written sentiment analysis reports from the customer service call recordings, and all translated text must be in English. Which combination of steps will meet these requirements? (Choose three.)

- ☐ Use Amazon Comprehend to translate the audio recordings into English.
Amazon Comprehend is for sentiment analysis, not audio translation.
- ☐ Use Amazon Lex to create the written sentiment analysis reports.
Amazon Lex is a service for building conversational interfaces, not sentiment analysis reports.
- ☐ Use Amazon Polly to convert the audio recordings into text.
Amazon Polly converts text to speech, not speech to text.
- ☒ Use Amazon Transcribe to convert the audio recordings in any language into text.
Amazon Transcribe can accurately convert audio recordings from multiple languages into text.
- ☒ Use Amazon Translate to translate text in any language to English.
Amazon Translate can translate text from various languages into English.
- ☒ Use Amazon Comprehend to create the sentiment analysis reports.
Amazon Comprehend is designed for sentiment analysis and can generate reports based on text input.

Overall Explanation:

The optimal solution involves using Amazon Transcribe to convert audio to text in any language. Then, use Amazon Translate to translate the text into English, and finally, use Amazon Comprehend to generate sentiment analysis reports from the English text. This combination allows the company to process calls in any language, translate them to English, and obtain sentiment analysis reports without needing to maintain custom ML models.

[View Discussions](#)[Ask AI](#)

Question 4

Not Attempted

Security



A company uses Amazon EC2 instances to host its internal systems. As part of a deployment operation, an administrator tries to use the AWS CLI to terminate an EC2 instance. However, the administrator receives a 403 (Access Denied) error message. The administrator is using an IAM role that has the following IAM policy attached:



Question illustration 1

What is the cause of the unsuccessful request?

- ☐ The EC2 instance has a resource-based policy with a Deny statement.
Resource-based policies can restrict access, but the question focuses on the IAM role's policy, making this less likely to be the primary cause.
- ☐ The principal has not been specified in the policy statement.
The provided policy applies to the IAM role assumed by the administrator. Principals are usually relevant for resource-based policies, not IAM policies.
- ☐ The "Action" field does not grant the actions that are required to terminate the EC2 instance.
The policy explicitly allows the `ec2:TerminateInstances` action. So, the action itself is not the issue.
- ☒ The request to terminate the EC2 instance does not originate from the CIDR blocks 192.0.2.0/24 or 203.0.113.0/24.
The IAM policy includes a Deny statement with a `NotIpAddress` condition. This means the action is denied unless the source IP address of the request is within the specified CIDR blocks. A 403 error

would occur if the request originated from an IP address outside of these ranges.

Overall Explanation:

The IAM policy includes a Deny statement that restricts the `ec2:TerminateInstances` action to specific source IP addresses (192.0.2.0/24 and 203.0.113.0/24). The 403 (Access Denied) error indicates that the request to terminate the EC2 instance originated from an IP address outside of the allowed CIDR blocks.

[View Discussions](#)[Ask AI](#)

Question 5

Not Attempted

Security



A company is conducting an internal audit to ensure that data in an Amazon S3 bucket associated with the company's AWS Lake Formation data lake does not contain sensitive customer or employee data. The company needs to discover personally identifiable information (PII) or financial information, including passport numbers and credit card numbers. Which solution will meet these requirements?

- ☐ Configure AWS Audit Manager on the account. Select the Payment Card Industry Data Security Standards (PCI DSS) for auditing.

AWS Audit Manager helps you continuously audit your AWS usage to simplify how you assess risk and compliance with regulations and industry standards. It does not directly discover sensitive data within S3 buckets.

- ☐ Configure Amazon S3 Inventory on the S3 bucket. Configure Amazon Athena to query the inventory.

Amazon S3 Inventory provides a list of your objects in an S3 bucket, but it does not identify sensitive data within those objects. While Athena can query the inventory, the inventory itself doesn't contain the PII or financial data.

- ☒ Configure Amazon Macie to run a data discovery job that uses managed identifiers for the required data types.

Amazon Macie is a data security service that uses machine learning and pattern matching to discover and help protect your sensitive data in AWS. It can identify PII and financial information using managed identifiers.

- ☐ Use Amazon S3 Select to run a report across the S3 bucket.

Amazon S3 Select enables you to retrieve subsets of data from an S3 object using SQL expressions. While S3 Select could be used to search for patterns, it's not designed for automated discovery and classification of sensitive data like Macie is.

Overall Explanation:

Amazon Macie is designed to discover, classify, and protect sensitive data in AWS. By configuring it to run a data discovery job with managed identifiers, the company can identify and classify PII and financial information like passport numbers and credit card numbers stored in the S3 bucket. Therefore, Macie is the best solution to meet the company's requirements.

[View Discussions](#)[Ask AI](#)

Question 6

Not Attempted

Storage



A company is facing storage capacity limitations with its on-premises servers hosting applications that rely on both block and NFS storage. To address this, they need a high-performing solution that supports local caching while avoiding significant application re-architecture. Which combination of actions would best meet these requirements?

- ☐ Mount Amazon S3 as a file system to the on-premises servers.
Mounting S3 directly as a file system may not provide the required performance or local caching capabilities.
- ☒ Deploy an AWS Storage Gateway file gateway to replace NFS storage.
The file gateway provides a seamless NFS interface with local caching to improve performance and reduce latency for file-based workloads.
- ☐ Deploy AWS Snowball Edge to provision NFS mounts to on-premises servers.
Snowball Edge is primarily for data transfer and edge computing, not for ongoing NFS storage replacement with local caching.
- ☒ Deploy an AWS Storage Gateway volume gateway to replace the block storage.
The volume gateway allows replacing block storage with cloud-backed storage and leverages local caching for performance optimization.

- ⊗ Deploy Amazon Elastic File System (Amazon EFS) volumes and mount them to on-premises servers.

While technically possible, using EFS on-premises typically requires a VPN or Direct Connect connection and may necessitate re-architecting applications, which goes against the stated requirement.

Overall Explanation:

Deploying an AWS Storage Gateway file gateway and a volume gateway allows the company to replace both NFS and block storage with cloud-backed solutions that support local caching. This combination meets the requirements for high performance and minimal application changes. The file gateway handles NFS storage with local caching, while the volume gateway handles block storage also with local caching. Other solutions either don't provide local caching or require significant re-architecting.

 View Discussions

 Ask AI

Question 7

Not Attempted

Networking



A company runs a service on Amazon EC2 instances in a private subnet of a VPC. This service frequently reads and writes large amounts of data to an Amazon S3 bucket within the same AWS Region. Currently, the service communicates with S3 via a NAT gateway located in a public subnet. To reduce data output costs and improve performance, the company seeks a more cost-effective solution for S3 access. Which of the following options BEST addresses these requirements?

- ⊗ Provision a dedicated EC2 NAT instance in the public subnet. Configure the route table for the private subnet to route all S3 traffic through the elastic network interface of this instance.

Using an EC2 NAT instance, while providing NAT functionality, does not inherently reduce data transfer costs for S3 traffic compared to a NAT gateway. It still routes traffic through the public internet, incurring data transfer charges.

- ⊗ Provision a dedicated EC2 NAT instance in the private subnet. Configure the route table for the public subnet to route all S3 traffic through the elastic network interface of this instance.

Placing the NAT instance in the private subnet and routing traffic through the public subnet is incorrect. This does not help reduce costs or improve performance.

- ✔ Provision a VPC gateway endpoint for S3. Configure the route table for the private subnet to use the gateway endpoint as the route for all S3 traffic.

A VPC gateway endpoint for S3 allows EC2 instances within the VPC to access S3 directly over the AWS internal network, bypassing the NAT gateway and the public internet. This eliminates data transfer costs associated with NAT gateways and provides improved performance.

- ⊗ Provision a second NAT gateway. Configure the route table for the private subnet to use this NAT gateway as the destination for all S3 traffic.

Adding another NAT gateway does not inherently reduce the cost of data transfer for S3 traffic. Data transferred through a NAT gateway is still subject to standard data transfer charges.

Overall Explanation:

A VPC gateway endpoint provides a direct, private connection to S3 within the AWS network, eliminating data transfer costs associated with routing traffic through a NAT gateway. Options A, B, and D still incur data transfer charges and do not provide the most cost-effective solution. Therefore, provisioning a VPC gateway endpoint (Option C) is the best solution to reduce data output costs when accessing S3 from EC2 instances within a private subnet.

 View Discussions

 Ask AI

Question 8

Not Attempted

Storage



A company stores high-resolution pictures in an Amazon S3 bucket. To simplify application code, the pictures are stored as the latest version of an S3 object. The company must retain only the two most recent versions of each picture and wants to minimize S3 costs with the LEAST operational overhead. Which solution meets these requirements?

- ✔ Use S3 Lifecycle policies to delete expired object versions and retain the two most recent versions.

S3 Lifecycle policies can automatically transition or expire objects based on age or other criteria. Configuring a policy to delete expired object versions and retain only the two most recent versions manages storage costs while maintaining the retention policy. This is automated and requires minimal operational overhead.

- ⊗ Use an AWS Lambda function to check for older versions and delete all but the two most recent versions.

While a Lambda function could achieve the desired outcome, it requires custom code and more operational overhead than using S3 Lifecycle policies.

- ⊗ Use S3 Batch Operations to delete noncurrent object versions and retain only the two most recent versions.

S3 Batch Operations is useful for large-scale changes across many objects. However, for ongoing version management, S3 Lifecycle policies are more suitable and require less operational overhead.

- ⊗ Deactivate versioning on the S3 bucket and retain the two most recent versions.

Deactivating versioning will not retain the two most recent versions; it will only retain the latest version. This does not meet the stated requirements of the question.

Overall Explanation:

S3 Lifecycle policies provide the most efficient and automated way to manage S3 object versions and reduce costs while requiring the least operational overhead. Option A correctly leverages this feature to delete expired object versions and retain the required number of recent versions. Other options involve higher operational overhead or don't fulfill the retention requirements.

 View Discussions

 Ask AI

Question 9

Not Attempted

Networking



A company currently utilizes a 1 Gbps AWS Direct Connect connection. Their average connection utilization is consistently below 10%. To reduce costs without sacrificing security, a solutions architect needs to recommend a more efficient solution. Which of the following options best addresses these requirements?

- ⊗ Set up a new 1 Gbps Direct Connect connection and share it with another AWS account.
Sharing a Direct Connect connection with another AWS account is not a secure solution.

- ⊗ Set up a new 200 Mbps Direct Connect connection directly through the AWS Management Console.

You cannot directly order a 200 Mbps Direct Connect connection through the AWS Management Console. Dedicated Connections have speeds of 1 Gbps, 10 Gbps and 100 Gbps

- ⊗ Contact an AWS Direct Connect Partner to order a 1 Gbps connection and share it with another AWS account.

Sharing a Direct Connect connection introduces potential security concerns.

- ✔ Contact an AWS Direct Connect Partner to order a 200 Mbps hosted connection for an existing AWS account.

Hosted connections through Direct Connect Partners offer the flexibility of lower bandwidth options like 200 Mbps, which is suitable for low utilization scenarios and cost optimization.

Overall Explanation:

Given the low utilization of the 1 Gbps Direct Connect connection, the most cost-effective and secure solution is to establish a 200 Mbps hosted connection through an AWS Direct Connect Partner. Hosted connections allow for speeds less than 1 Gbps, while directly managing connections through the AWS Management Console only allows for 1 Gbps or greater. Sharing a connection introduces unnecessary security risks. Therefore, only option D is correct.

 View Discussions

 Ask AI

Question 10

Not Attempted

Storage



A company is hosting a web application on AWS using a single Amazon EC2 instance that stores user-uploaded documents in an Amazon EBS volume. For better scalability and availability, the company duplicated the architecture and created a second EC2 instance and EBS volume in another Availability Zone, placing both behind an Application Load Balancer. After completing this change, users reported that, each time they refreshed the website, they could see one subset of their documents or the other, but never all of the documents at the same time. What should a solutions architect propose to ensure users see all of their documents at once?

- ⊗ Copy the data so both EBS volumes contain all the documents.

Copying data between EBS volumes doesn't guarantee data consistency and requires complex synchronization mechanisms.

- ⊗ Configure the Application Load Balancer to direct a user to the server with the documents.

The Application Load Balancer doesn't have the capability to track which documents reside on each server. This approach requires custom logic to manage document locations.

- ✓ Copy the data from both EBS volumes to Amazon EFS. Modify the application to save new documents to Amazon EFS.

Amazon EFS allows multiple EC2 instances to access a shared file system. By migrating the data to EFS, both instances can serve all documents, ensuring users always see all their files.

- ⊗ Configure the Application Load Balancer to send the request to both servers. Return each document from the correct server.

Application Load Balancers are designed to distribute traffic across healthy instances, not to split requests and merge responses. Implementing such logic would be complex and inefficient.

Overall Explanation:

Using Amazon EFS provides a shared storage solution, ensuring that both EC2 instances can access the same documents. This resolves the issue of users seeing different subsets of documents depending on which instance they are connected to. EBS volumes are tied to a single Availability Zone and do not provide a shared storage option across multiple instances in different AZs without complex and unreliable data replication strategies. Amazon EFS addresses these concerns.

 View Discussions

 Ask AI

Question 11

Not Attempted

Compute



A company runs a production workload on 1,000 Amazon EC2 Linux instances. The workload relies on third-party software. A critical security vulnerability has been identified in the third-party software, and the company needs to apply a patch to all EC2 instances as quickly as possible. Which solution should a solutions architect implement to meet these requirements?

- ⊗ Create an AWS Lambda function to apply the patch to all EC2 instances.

This is incorrect because Lambda functions are not designed to directly manage and patch EC2 instances. Lambda is better suited for event-driven tasks.

- ⊗ Configure AWS Systems Manager Patch Manager to apply the patch to all EC2 instances.

This is incorrect. Patch Manager is primarily designed for OS-level patches and might not recognize or handle third-party software patches effectively, especially for custom applications. Also, Patch Manager might require more configuration upfront.

- ⊗ Schedule an AWS Systems Manager maintenance window to apply the patch to all EC2 instances.

This is incorrect because maintenance windows are more suitable for scheduled tasks and less ideal for immediate remediation of critical vulnerabilities. Also, maintenance windows might not directly support patching of third-party applications.

- ✓ Use AWS Systems Manager Run Command to run a custom command that applies the patch to all EC2 instances.

This is correct. Run Command allows the execution of arbitrary commands or scripts across a fleet of EC2 instances. A custom script can be created to apply the third-party software patch quickly and efficiently.

Overall Explanation:

AWS Systems Manager Run Command provides the fastest and most direct way to apply a custom patch to all EC2 instances. Patch Manager is better suited for OS-level updates, and Lambda is not designed for EC2 instance management. Maintenance windows are suitable for scheduled tasks but not for critical, immediate patching.

 View Discussions

 Ask AI

Question 12

Not Attempted

Storage



A company has multiple Windows file servers on premises. The company wants to migrate and consolidate its files into an Amazon FSx for Windows File Server file system. File permissions must be preserved to ensure that access rights do not change. Which solutions will meet these requirements? (Choose two.)

- ✓ Deploy AWS DataSync agents on premises. Schedule DataSync tasks to transfer the data to the FSx for Windows File Server file system.

Deploying DataSync agents on-premises and scheduling DataSync tasks to transfer data to FSx for Windows File Server is a valid solution. DataSync preserves file permissions during the migration process.

- ⊗ Copy the shares on each file server into Amazon S3 buckets by using the AWS CLI. Schedule AWS DataSync tasks to transfer the data to the FSx for Windows File Server file system.

Copying files to S3 first will lose file permissions as S3 doesn't natively store Windows file permissions. Then using DataSync from S3 to FSx won't preserve original permissions.

- ⊗ Remove the drives from each file server. Ship the drives to AWS for import into Amazon S3. Schedule AWS DataSync tasks to transfer the data to the FSx for Windows File Server file system.

Removing drives and shipping them for import into S3 is not a viable option. S3 doesn't store native Windows file permissions, and this method is impractical and loses file permission information.

- ✓ Order an AWS Snowcone device. Connect the device to the on-premises network. Launch AWS DataSync agents on the device. Schedule DataSync tasks to transfer the data to the FSx for Windows File Server file system.

Using an AWS Snowcone device with DataSync agents to transfer data to FSx for Windows File Server preserves file permissions.

- ⊗ Order an AWS Snowball Edge Storage Optimized device. Connect the device to the on-premises network. Copy data to the device by using the AWS CLI. Ship the device back to AWS for import into Amazon S3. Schedule AWS DataSync tasks to transfer the data to the FSx for Windows File Server file system.

Copying the files via AWS CLI to Snowball Edge and then shipping it to S3 will lose file permissions before moving them to FSx.

Overall Explanation:

The correct answers are deploying AWS DataSync agents directly to FSx and using a Snowcone device with DataSync agents to transfer data. Both solutions utilize DataSync to ensure that file permissions are preserved during the migration to FSx for Windows File Server. Copying the files into S3 first will not preserve the original file permissions.

 View Discussions

 Ask AI

A company needs to ingest customer payment data into their Amazon S3 data lake. The company receives payment data every minute, and they require real-time analysis of the data before it is ingested into the data lake. Which solution provides the MOST operationally efficient way to meet these requirements?

- ⊗ Use Amazon Kinesis Data Streams to ingest data. Use AWS Lambda to analyze the data in real time.

While Kinesis Data Streams offers real-time ingestion and Lambda can provide real-time analytics, this solution is more complex and requires more operational overhead compared to Kinesis Data Firehose and Analytics.

- ⊗ Use AWS Glue to ingest data. Use Amazon Kinesis Data Analytics to analyze the data in real time.

AWS Glue is primarily an ETL service and is not designed for real-time ingestion. It's more suitable for batch processing.

- ✔ Use Amazon Kinesis Data Firehose to ingest data. Use Amazon Kinesis Data Analytics to analyze the data in real time.

Kinesis Data Firehose provides near real-time ingestion directly into S3. Kinesis Data Analytics allows for real-time analysis of the data stream. This is the most operationally efficient solution for this scenario.

- ⊗ Use Amazon API Gateway to ingest data. Use AWS Lambda to analyze the data in real time.

Using API Gateway and Lambda for this purpose is more complex and would involve more operational overhead than using Kinesis services. API Gateway is not optimized for high-volume data streaming.

Overall Explanation:

Amazon Kinesis Data Firehose is designed to ingest data directly into S3 with minimal operational overhead. Amazon Kinesis Data Analytics can then perform real-time analysis on the data stream as it's being ingested. This combination provides a streamlined and efficient solution compared to the other options. While Firehose has a near real-time latency, it fits the 'every minute' requirement. Alternatives involve more manual setup, coding, and operational maintenance.

 View Discussions

 Ask AI

Question 14

Not Attempted

Compute



A company runs a website that uses a content management system (CMS) on Amazon EC2. The CMS runs on a single EC2 instance and uses an Amazon Aurora MySQL Multi-AZ DB instance for the data tier. Website images are stored on an Amazon Elastic Block Store (Amazon EBS) volume that is mounted inside the EC2 instance. Which combination of actions should a solutions architect take to improve the performance and resilience of the website? (Choose two.)

- ☐ Move the website images into an Amazon S3 bucket that is mounted on every EC2 instance.
Mounting an S3 bucket directly to EC2 instances is not the most efficient approach for serving website images. While possible, it's generally better suited for object storage rather than frequent file access.
- ☐ Share the website images by using an NFS share from the primary EC2 instance. Mount this share on the other EC2 instances.
Using NFS from a single EC2 instance introduces a single point of failure and doesn't scale well for high-traffic websites.
- ☒ Move the website images onto an Amazon Elastic File System (Amazon EFS) file system that is mounted on every EC2 instance.
Amazon EFS is a scalable and fully managed file storage service that allows multiple EC2 instances to concurrently access the same file system. This provides improved performance and availability for website images.
- ☐ Create an Amazon Machine Image (AMI) from the existing EC2 instance. Use the AMI to provision new instances behind an Application Load Balancer as part of an Auto Scaling group. Configure the Auto Scaling group to maintain a minimum of two instances. Configure an accelerator in AWS Global Accelerator for the website.
While Auto Scaling and an Application Load Balancer improve resilience, AWS Global Accelerator is best suited for applications using Network Load Balancers, not Application Load Balancers as stated in the option. This option focuses less on image delivery optimization.
- ☒ Create an Amazon Machine Image (AMI) from the existing EC2 instance. Use the AMI to provision new instances behind an Application Load Balancer as part of an Auto Scaling group. Configure the Auto Scaling group to maintain a minimum of two instances. Configure an Amazon CloudFront distribution for the website.
Using Auto Scaling with an Application Load Balancer improves resilience by distributing traffic across multiple instances. Configuring an Amazon CloudFront distribution caches the website's

content (including images) at edge locations closer to users, resulting in lower latency and improved performance.

Overall Explanation:

Moving the images to Amazon EFS allows for shared storage across multiple instances which are behind an Application Load Balancer and part of an Auto Scaling group. Using CloudFront optimizes image delivery by caching the content at edge locations closer to users.

[View Discussions](#)[Ask AI](#)

Question 15

Not Attempted

Security



A company provides an infrastructure monitoring service and is developing a new feature to monitor customer AWS accounts. This feature requires calling AWS APIs within the customer accounts to gather information about Amazon EC2 instances and retrieve Amazon CloudWatch metrics. What is the MOST secure method for the company to gain access to these customer accounts?

- ☒ Ensure that the customers create an IAM role in their account with read-only EC2 and CloudWatch permissions and a trust policy that trusts the company's AWS account.
This approach utilizes cross-account IAM roles, granting temporary credentials via a trust policy, adhering to the principle of least privilege and avoiding long-term credentials storage.
- ☐ Create a serverless API that implements a token vending machine to provide temporary AWS credentials for a role with read-only EC2 and CloudWatch permissions within the company's account.
This option focuses on managing access to the company's resources, not securely accessing customer AWS accounts.
- ☐ Ensure that the customers create an IAM user in their account with read-only EC2 and CloudWatch permissions. Encrypt and store customer access and secret keys in a secrets management system.
Storing long-term access keys, even encrypted, is a security risk and should be avoided in favor of temporary credentials.

- ⊗ Ensure that the customers create an Amazon Cognito user in their account to use an IAM role with read-only EC2 and CloudWatch permissions. Encrypt and store the Amazon Cognito user and password in a secrets management system.

Amazon Cognito is designed for user authentication and authorization for web and mobile applications, not for service-to-service authentication across AWS accounts, and storing user credentials is a security risk.

Overall Explanation:

The most secure approach is for customers to create an IAM role with read-only permissions and a trust policy allowing the company's account to assume the role. This enables secure delegation and least privilege access through temporary credentials. Storing long-term credentials or using Cognito are less secure and inappropriate in this scenario.

 View Discussions

 Ask AI

Question 16

Not Attempted

Networking



A company needs to connect several VPCs in the us-east-1 Region that span hundreds of AWS accounts. The company's networking team has its own AWS account to manage the cloud network. What is the MOST operationally efficient solution to connect the VPCs?

- ⊗ Set up VPC peering connections between each VPC. Update each associated subnet's route table.

VPC peering becomes complex and difficult to manage when dealing with a large number of VPCs. It creates a mesh network of connections, which increases operational overhead.

- ⊗ Configure a NAT gateway and an internet gateway in each VPC to connect each VPC through the internet.

Using NAT gateways and internet gateways for inter-VPC communication is not efficient or secure. It exposes the VPCs to the public internet and adds unnecessary network hops.

- ✔ Create an AWS Transit Gateway in the networking team's AWS account. Configure static routes from each VPC.

AWS Transit Gateway provides a centralized hub for connecting multiple VPCs, simplifying network management and reducing operational complexity. It is the most efficient solution for connecting a large number of VPCs across multiple accounts.

- ⊗ Deploy VPN gateways in each VPC. Create a transit VPC in the networking team's AWS account to connect to each VPC.

Using VPN gateways and a transit VPC adds unnecessary complexity and cost. VPN gateways are typically used for connecting to on-premises networks, not for connecting VPCs within the same region.

Overall Explanation:

AWS Transit Gateway (TGW) is the most operationally efficient solution. It acts as a hub, centralizing the management of connections between multiple VPCs across different AWS accounts. TGW reduces the complexity associated with managing a large mesh of VPC peering connections or routing traffic through the public internet. Static routes are needed to direct traffic to the TGW.

 View Discussions

 Ask AI

Question 17

Not Attempted

Compute



A company runs nightly batch jobs to process data using Amazon EC2 instances within an Auto Scaling group utilizing On-Demand billing. These jobs run between 12:00 AM and 06:00 AM local time daily. If a job fails on an instance, another instance automatically reprocesses it. What is the MOST cost-effective solution to provision EC2 instances for these requirements?

- ⊗ Purchase a 1-year Savings Plan for Amazon EC2 covering the instance family used by the Auto Scaling group.

Savings Plans are better suited for long-running, steady-state workloads and may not be the most cost-effective for short, nightly batch jobs.

- ⊗ Purchase a 1-year Reserved Instance for the specific instance type and operating system of the instances in the Auto Scaling group.

Reserved Instances are suitable for long-running workloads. Short, nightly batch jobs benefit more from a different approach.

- ✔ Create a new launch template for the Auto Scaling group. Configure the instances to use Spot Instances and set a policy to scale out based on CPU utilization.

Spot Instances provide significant cost savings compared to On-Demand instances and are suitable for fault-tolerant workloads like these batch jobs, where reprocessing failed jobs is acceptable. Scaling out based on CPU usage ensures adequate resources when needed.

- ⊗ Create a new launch template for the Auto Scaling group. Increase the instance size and set a policy to scale out based on CPU usage.

Increasing instance size may not be cost-effective if the current size is sufficient. Scaling out based on CPU might add cost unnecessarily compared to Spot Instances.

Overall Explanation:

Spot Instances are the most cost-effective choice for batch jobs that run for a specific period each day and can tolerate interruptions, as failed jobs can be reprocessed.

Configuring the Auto Scaling group to use Spot Instances and scale out based on CPU usage ensures adequate capacity while minimizing costs. Savings Plans and Reserved Instances are designed for long-term, stable workloads and are not ideal for this use case.

 View Discussions

 Ask AI

Question 18

Not Attempted

Storage



A popular social media platform anticipates a surge in photo uploads during global events. To ensure seamless performance and scalability, they need a solution that can efficiently handle the expected increase in upload traffic. Which approach offers the MOST scalable solution for uploading user photos?

- ⊗ Route file uploads from users' browsers to the application servers, and then transfer the files to an Amazon S3 bucket.

This approach places the application servers in the data path, making them a bottleneck for upload traffic and hindering scalability.

- ⊗ Implement an AWS Storage Gateway file gateway, enabling users to upload files directly from their browsers to the gateway.

AWS Storage Gateway is designed for hybrid cloud storage and isn't suitable for handling large volumes of concurrent uploads from internet users due to its limited scalability and intended use case.

- ✔ Generate Amazon S3 presigned URLs within the application. Allow users to upload files directly from their browsers into an S3 bucket using these URLs.

Using presigned URLs, users can upload files directly to S3, bypassing the application servers. This offloads upload processing, significantly enhancing scalability and minimizing the load on the

application. S3 is designed to handle massive amounts of uploads, making this the most scalable solution.

- ⊗ Provision an Amazon Elastic File System (Amazon EFS) file system. Configure the application to facilitate users uploading files directly to the EFS file system.

Amazon EFS is designed for shared file storage within an EC2 environment, not for handling direct uploads from a large number of internet users. It lacks the scalability and security features necessary for this use case.

Overall Explanation:

The MOST scalable solution is to generate Amazon S3 presigned URLs and allow users to upload files directly to Amazon S3. This approach offloads the upload processing to S3, which is designed for massive scalability and eliminates the application servers as a bottleneck. It leverages the client-side capabilities for file uploads, resulting in a highly efficient and scalable system.

 View Discussions

 Ask AI

Question 19

Not Attempted

Databases



A global travel company with users worldwide currently operates its web application and reservation database from a single data center in North America. To improve user experience and reduce latency for its international users, the company plans to expand its application deployment across multiple AWS Regions. A critical requirement is to maintain a single, globally consistent primary reservation database with average latency of less than 1 second for all updates. The company wants to maintain separate web platform deployments across multiple regions using the single primary reservation database. Which solution should a solutions architect recommend?

- ⊗ Convert the application to use Amazon DynamoDB. Use a global table for the central reservation table. Use the correct Regional endpoint in each Regional deployment.

DynamoDB global tables provide multi-region, multi-master replication. While they offer global consistency, Aurora provides lower latency.

- ✓ Migrate the database to an Amazon Aurora MySQL database. Deploy Aurora Read Replicas in each Region. Use the correct Regional endpoint in each Regional deployment for access to the database.

Aurora Global Database provides low latency replication across regions, with a single primary instance for writes and read replicas for localized reads. This solution satisfies the latency and consistency requirements.

- ⊗ Migrate the database to an Amazon RDS for MySQL database. Deploy MySQL read replicas in each Region. Use the correct Regional endpoint in each Regional deployment for access to the database.

RDS MySQL read replicas provide asynchronous replication, which may not meet the strict latency requirement of less than 1 second for updates. In addition, manual failover is not efficient.

- ⊗ Migrate the application to an Amazon Aurora Serverless database. Deploy instances of the database to each Region. Use the correct Regional endpoint in each Regional deployment to access the database. Use AWS Lambda functions to process event streams in each Region to synchronize the databases.

This solution introduces complexity with Lambda functions for synchronization and may not guarantee sub-second latency for updates. Aurora Serverless does not directly provide multi-region replication.

Overall Explanation:

Aurora Global Database (option B) is the best solution because it maintains a single primary database and provides a low-latency, globally consistent solution. By using read replicas in each region, the application can serve localized read requests and minimize cross-region latency. The average latency is less than 1 second which will satisfy the requirement. DynamoDB might have higher latency. RDS read replicas offer async replication.

 View Discussions

 Ask AI

Question 20

Not Attempted

Storage



A company has migrated multiple Windows Server workloads to Amazon EC2 instances in the us-west-1 Region. The company performs manual backups to create images when needed. In the event of a natural disaster in us-west-1, the company needs to quickly recover workloads in us-west-2 with a maximum data loss of 24 hours. The company also requires automated EC2 instance backups with minimal administrative effort. Which solutions meet these requirements?

- ⊗ Create an EC2-backed AMI lifecycle policy to create backups based on tags. Schedule the backup to run twice daily. Copy the image on demand.

Copying AMIs on demand requires manual intervention, which increases administrative overhead and doesn't meet the automation requirement.

- ✓ Create an EC2-backed AMI lifecycle policy to create backups based on tags. Schedule the backup to run twice daily. Configure the copy to the us-west-2 Region.

This option leverages AMI lifecycle policies to automate backups based on tags and automatically copies them to the us-west-2 region. This satisfies the automation and recovery requirements with minimal effort.

- ⊗ Create backup vaults in us-west-1 and us-west-2 using AWS Backup. Create a backup plan for the EC2 instances based on tag values. Create an AWS Lambda function to run as a scheduled job to copy the backup data to us-west-2.

Using a Lambda function to copy the backup data introduces unnecessary complexity and administrative overhead. AWS Backup can directly handle the cross-region copy.

- ✓ Create a backup vault using AWS Backup. Use AWS Backup to create a backup plan for the EC2 instances based on tag values. Define the destination for the copy as us-west-2. Specify the backup schedule to run twice daily.

This solution uses AWS Backup to create a backup plan with a defined destination region, automating the backup and replication process. This provides a centralized backup management solution with minimal admin overhead.

- ⊗ Create a backup vault using AWS Backup. Use AWS Backup to create a backup plan for the EC2 instances based on tag values. Specify the backup schedule to run twice daily. Copy on demand to us-west-2.

Similar to Option A, copying on demand requires manual intervention which isn't suitable for automation and minimal administrative effort.

Overall Explanation:

Options B and D provide the least administrative effort to automate backups and replicate them to the us-west-2 region. Option B uses EC2 AMI lifecycle policies for automated AMI creation and cross-region copying. Option D utilizes AWS Backup, which simplifies and centralizes backup management for EC2 instances with cross-region replication. Both options avoid manual intervention and ensure that data is backed up and replicated to the recovery region within the required time frame.

[View Discussions](#)[Ask AI](#)

Question 21

Not Attempted

Networking



A company's two-tier image processing application is experiencing performance issues due to a flood of illegitimate requests from a few IP addresses. The application uses two Availability Zones, each with a public and private subnet. The web tier utilizes an Application Load Balancer (ALB) in the public subnets, while the application tier's EC2 instances reside in the private subnets. A solutions architect needs to quickly mitigate the performance problem while a long-term solution is developed. Which of the following actions should the solutions architect recommend?

- ☐ Modify the inbound security group for the web tier. Add a deny rule for the IP addresses that are consuming resources.

Security groups operate at the instance level and cannot explicitly deny traffic; they only allow traffic. Therefore, this option is incorrect.

- ☒ Modify the network ACL for the web tier subnets. Add an inbound deny rule for the IP addresses that are consuming resources.

Network ACLs operate at the subnet level and can explicitly deny traffic. By adding a deny rule for the offending IP addresses to the web tier subnets' ACL, the illegitimate traffic is blocked before it reaches the ALB or web servers, improving performance.

- ☐ Modify the inbound security group for the application tier. Add a deny rule for the IP addresses that are consuming resources.

Security groups operate at the instance level and cannot explicitly deny traffic; they only allow traffic. Additionally, the attack is focused on the web tier, not the application tier, making this option ineffective.

- ☐ Modify the network ACL for the application tier subnets. Add an inbound deny rule for the IP addresses that are consuming resources.

The illegitimate traffic is targeting the web tier, not the application tier. Modifying the application tier's network ACL would not prevent the malicious traffic from reaching the ALB and the web servers.

Overall Explanation:

The most effective way to immediately mitigate the performance issues caused by the illegitimate traffic is to modify the network ACL for the web tier subnets. By adding an

inbound deny rule for the offending IP addresses, the traffic is blocked at the subnet level before it can reach the ALB and the web servers, improving performance. Security groups cannot deny traffic, and the application tier is not the target of the attack.

[View Discussions](#)[Ask AI](#)

Question 22

Not Attempted

Serverless



A company is developing an application that provides order shipping statistics for retrieval by a REST API. The company wants to extract the shipping statistics, organize the data into an easy-to-read HTML format, and send the report to several email addresses at the same time every morning. Which combination of steps should a solutions architect take to meet these requirements? (Choose two.)

- ☐ ☒ **Configure the application to send the data to Amazon Kinesis Data Firehose.**
Kinesis Data Firehose is designed for streaming data and is not suitable for scheduling and formatting reports.
- ☒ **Use Amazon Simple Email Service (Amazon SES) to format the data and to send the report by email.**
Amazon SES can format the data into HTML and send the formatted report to multiple recipients via email.
- ☐ ☒ **Create an Amazon EventBridge (Amazon CloudWatch Events) scheduled event that invokes an AWS Glue job to query the application's API for the data.**
AWS Glue is more suited for complex ETL tasks involving large datasets, which is more than needed for querying an API and formatting the response. Also, Glue has no native functionality to query REST APIs.
- ☒ **Create an Amazon EventBridge (Amazon CloudWatch Events) scheduled event that invokes an AWS Lambda function to query the application's API for the data.**
Amazon EventBridge can schedule an AWS Lambda function to run at a specific time, allowing the company to query the API every morning.
- ☐ ☒ **Store the application data in Amazon S3. Create an Amazon Simple Notification Service (Amazon SNS) topic as an S3 event destination to send the report by email.**
Storing data in Amazon S3 and using Amazon SNS doesn't address the requirement to format the data into an HTML report. SNS is better suited for sending basic notifications and the schedule

event is not available.

Overall Explanation:

The correct combination of steps is to use Amazon EventBridge to schedule an AWS Lambda function to query the application's API and use Amazon SES to format the data into an HTML email and send it. EventBridge provides the scheduling capability to run the process every morning. Lambda provides compute to query the API, while SES handles the HTML formatting and email sending.

[View Discussions](#)[Ask AI](#)

Question 23

Not Attempted

Networking



A global marketing company's applications run in the ap-southeast-2 Region and the eu-west-1 Region. Applications in a VPC in eu-west-1 need to communicate securely with databases in a VPC in ap-southeast-2. Which network design meets these requirements?

- ☐ Create a VPC peering connection between the eu-west-1 VPC and the ap-southeast-2 VPC. Create an inbound rule in the eu-west-1 application security group that allows traffic from the database server IP addresses in the ap-southeast-2 security group.
This option is incorrect because you cannot directly reference security group IP addresses in another security group's rule within a different region.
- ☐ Configure a VPC peering connection between the ap-southeast-2 VPC and the eu-west-1 VPC. Update the subnet route tables. Create an inbound rule in the ap-southeast-2 database security group that references the security group ID of the application servers in eu-west-1.
This option is incorrect because you cannot reference the security group of a peer VPC that's in a different Region. Instead, use the CIDR block of the peer VPC.
- ☒ Configure a VPC peering connection between the ap-southeast-2 VPC and the eu-west-1 VPC. Update the subnet route tables. Create an inbound rule in the ap-southeast-2 database security group that allows traffic from the eu-west-1 application server IP addresses.
This option is correct. It establishes a VPC peering connection and updates the subnet route tables to enable cross-region communication. It also creates an inbound rule in the database security group, allowing traffic from the application server IP addresses in the eu-west-1 VPC. Although this

is a valid approach, it is less dynamic than referencing a security group, since if an IP address changes, it needs to be updated in the rule manually.

- ⊗ Create a transit gateway with a peering attachment between the eu-west-1 VPC and the ap-southeast-2 VPC. After the transit gateways are properly peered and routing is configured, create an inbound rule in the database security group that references the security group ID of the application servers in eu-west-1.

Using Transit Gateway is an over-engineered solution for just connecting two VPCs. Also, you cannot reference the security group of a peer VPC that's in a different Region. Instead, use the CIDR block of the peer VPC.

Overall Explanation:

The correct answer is C. VPC Peering enables network connectivity between two VPCs. Route tables must be configured to allow for traffic to flow between the VPCs. To secure communication, the security group of the database in ap-southeast-2 should be configured to allow access from the IP addresses in eu-west-1.

[View Discussions](#)[Ask AI](#)

Question 24

Not Attempted

Databases



A company is developing software that uses a PostgreSQL database schema. The company needs to configure multiple development environments and databases for the company's developers. On average, each development environment is used for half of the 8-hour workday. Which solution will meet these requirements MOST cost-effectively?

- ⊗ Configure each development environment with its own Amazon Aurora PostgreSQL database.

While Aurora offers high performance, it's generally more expensive than RDS and less cost-effective when not fully utilized, especially without considering on-demand or serverless options.

- ⊗ Configure each development environment with its own Amazon RDS for PostgreSQL Single-AZ DB instances.

RDS Single-AZ is cheaper than Aurora, but the question does not mention to stop the DB when not in use, which would lead to higher costs.

- ✔ Configure each development environment with its own Amazon Aurora On-Demand PostgreSQL-Compatible database.

Aurora On-Demand (Serverless) allows you to pay only for the hours used. Given the low weekly usage (4 hours per day), this option will be the most cost-effective.

- ⊗ Configure each development environment with its own Amazon S3 bucket by using Amazon S3 Object Select.

Amazon S3 Object Select is used for querying data stored in S3, not for running a PostgreSQL database. This is not a suitable solution for development environments requiring a relational database.

Overall Explanation:

Amazon Aurora On-Demand PostgreSQL-Compatible databases (Aurora Serverless) are the most cost-effective solution. Since each development environment is only used for half of the 8-hour workday, Aurora Serverless allows you to pay only for the compute used during those hours, which results in significant cost savings compared to provisioned RDS or Aurora instances that run continuously. Using S3 Object Select is irrelevant to the question of providing PostgreSQL development databases.

 View Discussions

 Ask AI

Question 25

Not Attempted

Storage



A company uses AWS Organizations with resources tagged by account. The company also uses AWS Backup to back up its AWS infrastructure resources. The company needs to back up *all* AWS resources with the LEAST operational overhead. Which solution will meet these requirements?

- ✔ Use AWS Config to identify all untagged resources. Tag the identified resources programmatically. Use tags in the backup plan.

AWS Config can identify untagged resources. Programmatic tagging and tag-based backup plans minimize operational overhead and ensure all resources are backed up.

- ⊗ Use AWS Config to identify all resources that are not running. Add those resources to the backup vault.

Backing up only non-running resources does not meet the requirement of backing up all resources. Additionally, 'not running' is not a useful criteria for backup.

- ⊗ Require all AWS account owners to review their resources to identify the resources that need to be backed up.

This is the most operationally expensive approach and is prone to human error.

- ⊗ Use Amazon Inspector to identify all noncompliant resources.

Amazon Inspector identifies security vulnerabilities and compliance issues, not resources that need backups. Tagging those identified is also not its function.

Overall Explanation:

The most efficient solution uses AWS Config to continuously evaluate resource configurations and identify untagged resources. These resources can then be programmatically tagged and a backup plan leveraging these tags will automatically backup the tagged resources as they are created, minimizing manual intervention. This approach ensures that all resources are included in the backup process while minimizing operational overhead.

[View Discussions](#)[Ask AI](#)

Question 26

Not Attempted

Serverless



A social media company needs to allow its users to upload images to an application hosted in the AWS Cloud. The company needs a solution that automatically resizes the images for display on various devices. The application experiences unpredictable traffic patterns. The company requires a highly available and scalable solution. Which approach best meets these requirements?

- ✔ Create a static website hosted in Amazon S3 that invokes AWS Lambda functions to resize the images and store the images in an Amazon S3 bucket.

This solution leverages S3 for hosting, providing scalability and availability, and uses Lambda for automatic image resizing on upload, which dynamically scales with traffic. S3 provides durable storage for the images.

- ⊗ Create a static website hosted in Amazon CloudFront that invokes AWS Step Functions to resize the images and store the images in an Amazon RDS database.

Using Amazon RDS for storing images is not efficient. Step functions is a workflow service and not suitable for image processing.

- ⊗ Create a dynamic website hosted on a web server that runs on an Amazon EC2 instance. Configure a process that runs on the EC2 instance to resize the images and store the images in an Amazon S3 bucket.

While EC2 can resize images, managing the scalability and availability of the web server and resizing process would require additional configuration and management effort compared to a serverless approach.

- ⊗ Create a dynamic website hosted on an automatically scaling Amazon Elastic Container Service (Amazon ECS) cluster that creates a resize job in Amazon Simple Queue Service (Amazon SQS). Set up an image-resizing program that runs on an Amazon EC2 instance to process the resize jobs.

This is overly complex. A simpler solution using Lambda would be more efficient.

Overall Explanation:

The most suitable solution is to use a static website hosted in Amazon S3 that triggers AWS Lambda functions to resize the images and store them back in an Amazon S3 bucket. This serverless approach provides high availability, automatic scalability, and cost-effectiveness, perfectly aligning with the unpredictable traffic patterns and the need for automatic image resizing. S3 provides durable and scalable storage, and Lambda handles the image resizing without the need for managing servers.

[View Discussions](#)[Ask AI](#)

Question 27

Not Attempted

Networking



A company is migrating a microservices application from Amazon EC2 instances to an Amazon Elastic Kubernetes Service (Amazon EKS) cluster for improved scalability. For security compliance, the EKS control plane is configured with endpoint private access set to true and endpoint public access set to false. The data plane nodes are deployed in private subnets. After deployment, the company receives error notifications indicating that the nodes cannot join the cluster. What is the MOST likely solution to resolve this issue and allow the nodes to successfully join the EKS cluster?

- ⊗ Grant the required permissions in AWS Identity and Access Management (IAM) to the AmazonEKSNodeRole IAM role.

While correct IAM permissions are necessary for nodes to operate within the cluster, this doesn't directly address the connectivity issue caused by a private control plane endpoint and nodes in private subnets. The nodes need a way to reach the control plane API privately.

✔ Create interface VPC endpoints to allow nodes to access the control plane.

This is the MOST likely solution. With a private control plane endpoint and nodes in private subnets, the nodes need a private network path to communicate with the control plane. Interface VPC endpoints (specifically for EKS) provide this private connectivity, enabling the nodes to join the cluster.

✘ Recreate nodes in the public subnet. Restrict security groups for EC2 nodes.

Recreating nodes in public subnets defeats the purpose of having a private control plane and private data plane for security. This is not the best practice.

✘ Allow outbound traffic in the security group of the nodes.

While allowing outbound traffic is generally necessary, it doesn't solve the core problem of how the nodes in private subnets can reach the private control plane endpoint. Outbound traffic needs a route, and in this case, that route should be via VPC endpoints, not a public internet gateway.

Overall Explanation:

The most likely solution is to create interface VPC endpoints to allow the nodes in the private subnets to communicate with the EKS control plane, which has public access disabled. This enables private network communication between the data plane nodes and the control plane API.

 View Discussions

 Ask AI

Question 28

Not Attempted

Databases



A company is migrating an on-premises application to AWS. The company wants to use Amazon Redshift as a data warehouse solution. Which of the following use cases are suitable for Amazon Redshift in this scenario? (Choose three.)

✘ Supporting data APIs to access data with traditional, containerized, and event-driven applications

Amazon Redshift provides a Data API that can be used to access data from various types of applications.

- ✓ Supporting client-side and server-side encryption
Amazon Redshift supports both client-side and server-side encryption for improved data security.
- ✓ Building analytics workloads during specified hours and when the application is not active
Amazon Redshift is optimized for running complex analytic queries against large datasets, making it suitable for running workloads during off-peak hours.
- ✗ Caching data to reduce the pressure on the backend database
While Redshift does have caching, it is not its primary purpose. Other services like ElastiCache or DAX are better suited for general caching.
- ✓ Scaling globally to support petabytes of data and tens of millions of requests per minute
Amazon Redshift is designed to handle petabytes of data and can deliver fast query performance for large datasets, scaling for analytics workloads.
- ✗ Creating a secondary replica of the cluster by using the AWS Management Console
Redshift doesn't directly create replicas via the AWS Management Console in the way this option implies. DR clusters are created from snapshots and restored to another region.

Overall Explanation:

Amazon Redshift is a data warehouse solution optimized for analytics. Therefore, supporting encryption (client-side and server-side), handling analytics workloads during off-peak hours, and scaling to handle large amounts of data and high query volumes for analytics purposes are all suitable use cases. Data APIs are supported by Redshift but are not its primary function. Caching isn't the primary purpose of Redshift. Creating replicas is not done directly via the console.

[View Discussions](#)[Ask AI](#)

Question 29

Not Attempted

[Compute](#)

A company provides an API interface to customers for retrieving their financial information. The company anticipates a surge in requests during peak usage times of the year. To ensure customer satisfaction, the API must respond consistently with low latency. The company seeks a

compute solution for the API with the LEAST operational overhead. Which solution best meets these requirements?

- ☐ Use an Application Load Balancer and Amazon Elastic Container Service (Amazon ECS).
ECS requires managing containers and scaling, adding operational overhead. While scalable, it's not the *least* overhead option.
- ☒ Use Amazon API Gateway and AWS Lambda functions with provisioned concurrency.
API Gateway and Lambda offer serverless compute, reducing operational overhead. Provisioned concurrency minimizes cold starts, ensuring consistent low latency during peak periods.
- ☐ Use an Application Load Balancer and an Amazon Elastic Kubernetes Service (Amazon EKS) cluster.
EKS requires significant operational overhead in managing the Kubernetes cluster. This is a complex solution and not the least overhead.
- ☐ Use Amazon API Gateway and AWS Lambda functions with reserved concurrency.
Reserved concurrency guarantees available execution capacity for functions, but provisioned concurrency pre-warms instances, further reducing cold starts and latency.

Overall Explanation:

Amazon API Gateway and AWS Lambda with provisioned concurrency provide the least operational overhead. API Gateway handles API requests and integrates with Lambda. Lambda automatically scales compute without managing servers. Provisioned concurrency ensures consistent low latency by pre-warming function instances, avoiding cold starts. ECS and EKS require managing containers and clusters, increasing operational overhead. Provisioned concurrency offers benefits of low latency compared to reserved concurrency.

[View Discussions](#)[Ask AI](#)

Question 30

Not Attempted

[Monitoring and Logging](#)

A company wants to send all AWS Systems Manager Session Manager logs to an Amazon S3 bucket for archival purposes. Which solution will meet this requirement with the MOST operational efficiency?

- ✓ Enable S3 logging in the Systems Manager console. Choose an S3 bucket to send the session data to.

Enabling S3 logging in the Systems Manager console is the most direct and efficient way to send Session Manager logs to an S3 bucket. It requires minimal configuration and overhead.

- ✗ Install the Amazon CloudWatch agent. Push all logs to a CloudWatch log group. Export the logs to an S3 bucket from the group for archival purposes.

While this option can work, it involves additional steps and complexity compared to directly enabling S3 logging in Session Manager. It introduces CloudWatch as an intermediary service, increasing operational overhead.

- ✗ Create a Systems Manager document to upload all server logs to a central S3 bucket. Use Amazon EventBridge to run the Systems Manager document against all servers that are in the account daily.

This solution is overly complex and inefficient. Creating a custom Systems Manager document and scheduling it with EventBridge adds unnecessary overhead for a task that can be accomplished with a built-in feature.

- ✗ Install an Amazon CloudWatch agent. Push all logs to a CloudWatch log group. Create a CloudWatch logs subscription that pushes any incoming log events to an Amazon Kinesis Data Firehose delivery stream. Set Amazon S3 as the destination.

This option is the most complex and least efficient. It involves multiple services (CloudWatch, Kinesis Data Firehose) and a significant amount of configuration, leading to high operational overhead.

Overall Explanation:

Enabling S3 logging directly within the Systems Manager console is the most operationally efficient solution. It's a built-in feature that directly sends session logs to an S3 bucket with minimal configuration and management overhead. The other options involve additional services and configurations, increasing complexity and operational burden. Using the built-in feature provides simplicity, direct integration, automated flow, easy management, cost-effectiveness, and minimal overhead.

[View Discussions](#)[Ask AI](#)

Question 31

Not Attempted

Databases



An application utilizes an Amazon RDS MySQL DB instance. The RDS database is experiencing low disk space. A solutions architect needs to increase the disk space with minimal effort and without incurring any downtime. Which solution best meets these requirements?

☒ Enable storage autoscaling in RDS.

Enabling storage autoscaling allows RDS to automatically adjust the storage capacity based on the application's needs. When storage usage exceeds a predefined threshold, RDS increases the allocated storage without manual intervention or downtime.

☐ Increase the RDS database instance size.

Increasing the instance size doesn't directly address disk space issues and might not be the most efficient solution. It might involve downtime depending on the instance type modification.

☐ Change the RDS database instance storage type to Provisioned IOPS.

Changing the storage type focuses on IOPS performance rather than disk space. Also, changing the storage type may incur downtime.

☐ Back up the RDS database, increase the storage capacity, restore the database, and stop the previous instance.

This approach is unnecessarily complex and involves significant downtime during the backup and restore processes.

Overall Explanation:

Enabling storage autoscaling is the most straightforward and efficient solution to increase disk space on an RDS instance without downtime. RDS Storage Auto Scaling automatically scales storage capacity in response to growing database workloads, with zero downtime. It avoids manual intervention and ensures the database has sufficient disk space. The other options involve either unnecessary complexity, potential downtime, or a focus on performance rather than storage capacity.

 View Discussions

 Ask AI

Question 32

Not Attempted

Application Integration



A consulting company provides professional services to customers worldwide. The company provides solutions and tools for customers to expedite gathering and analyzing data on AWS.

The company needs to centrally manage and deploy a common set of solutions and tools for customers to use for self-service purposes. Which solution will meet these requirements?

- ☐ Create AWS CloudFormation templates for the customers.
CloudFormation allows infrastructure as code, but doesn't offer self-service catalog capabilities.
- ☒ Create AWS Service Catalog products for the customers.
AWS Service Catalog allows you to create and manage catalogs of IT services that can be deployed within your organization. It enables self-service provisioning of standardized solutions and tools.
- ☐ Create AWS Systems Manager templates for the customers.
AWS Systems Manager is a management solution for resources and doesn't directly facilitate self-service catalogs.
- ☐ Create AWS Config items for the customers.
AWS Config assesses, audits, and evaluates configurations and does not enable self-service deployment of solutions.

Overall Explanation:

AWS Service Catalog allows you to create and manage catalogs of IT services that can be deployed within your organization. With Service Catalog, you can define a standardized set of products (solutions and tools in this case) that customers can self-service provision. By creating Service Catalog products, you can control and enforce the deployment of approved and validated solutions and tools. Key advantages include centralized management, self-service access, standardization, access control, and reporting.

 [View Discussions](#)

 [Ask AI](#)

Question 33

Not Attempted

[Storage](#)



A company is migrating an on-premises application to AWS. This application generates output files with sizes ranging from tens of gigabytes to hundreds of terabytes. The application data needs to be stored in a standard file system structure. The company requires a solution that automatically scales, is highly available, and minimizes operational overhead. Which solution best meets these requirements?

- ⊗ Migrate the application to run as containers on Amazon Elastic Container Service (Amazon ECS). Use Amazon S3 for storage.

Amazon S3 is object storage and doesn't provide a standard file system structure. While ECS manages containers, it doesn't inherently provide a file system.

- ⊗ Migrate the application to run as containers on Amazon Elastic Kubernetes Service (Amazon EKS). Use Amazon Elastic Block Store (Amazon EBS) for storage.

Amazon EBS is block storage, not a file system, and requires manual management and scaling, increasing operational overhead. EKS also requires more management than other options.

- ✓ Migrate the application to Amazon EC2 instances in a Multi-AZ Auto Scaling group. Use Amazon Elastic File System (Amazon EFS) for storage.

Amazon EFS provides a scalable and fully managed file system that can be easily mounted to multiple EC2 instances. It scales automatically, offers high availability across multiple Availability Zones, and minimizes operational overhead.

- ⊗ Migrate the application to Amazon EC2 instances in a Multi-AZ Auto Scaling group. Use Amazon Elastic Block Store (Amazon EBS) for storage.

Amazon EBS is block storage, not a file system. It does not scale automatically and has a scope limited to one Availability Zone.

Overall Explanation:

Amazon EFS is the best choice because it's a fully managed, scalable file system that meets the requirements for a standard file system structure, automatic scaling, high availability, and minimal operational overhead. Options A and B utilize object or block storage, which don't satisfy the standard file system requirement. Option D uses EBS, which does not scale automatically.

 View Discussions

 Ask AI

Question 34

Not Attempted

Databases



A company is designing a new web application that will run on Amazon EC2 Instances. The application will use Amazon DynamoDB for backend data storage. The application traffic will be unpredictable. The company expects that the application read and write throughput to the database will be moderate to high. The company needs to scale in response to application

traffic. Which DynamoDB table configuration will meet these requirements MOST cost-effectively?

- ☐ ⓧ Configure DynamoDB with provisioned read and write by using the DynamoDB Standard table class. Set DynamoDB auto scaling to a maximum defined capacity.
This option is not ideal because the traffic is unpredictable. Provisioned capacity is best suited for predictable workloads, even with auto-scaling.
- ☒ ✓ Configure DynamoDB in on-demand mode by using the DynamoDB Standard table class.
This is the most cost-effective solution. On-demand mode automatically adjusts to varying traffic patterns, which is ideal for unpredictable workloads. The Standard table class is suitable for moderate to high read and write throughput, and on-demand mode bills based on actual usage.
- ☐ ⓧ Configure DynamoDB with provisioned read and write by using the DynamoDB Standard Infrequent Access (DynamoDB Standard-IA) table class. Set DynamoDB auto scaling to a maximum defined capacity.
This option is not suitable. Standard-IA is designed for infrequently accessed data and is not appropriate for moderate to high traffic.
- ☐ ⓧ Configure DynamoDB in on-demand mode by using the DynamoDB Standard Infrequent Access (DynamoDB Standard-IA) table class.
This option is not suitable. Standard-IA is designed for infrequently accessed data and is not appropriate for moderate to high traffic, regardless of using on-demand mode.

Overall Explanation:

The best solution is to configure DynamoDB in on-demand mode with the DynamoDB Standard table class. On-demand mode is designed for unpredictable traffic patterns, as DynamoDB automatically adjusts capacity and you pay only for what you use. DynamoDB Standard is appropriate for moderate to high traffic. Standard-IA is for infrequent access, so options using it are incorrect. Provisioned capacity, even with autoscaling, is less cost-effective for unpredictable workloads because it's harder to tune the capacity ahead of the traffic.

[View Discussions](#)[Ask AI](#)

A retail company has several businesses. The IT team for each business manages its own AWS account. Each team account is part of an organization in AWS Organizations. Each team monitors its product inventory levels in an Amazon DynamoDB table in the team's own AWS account. The company is deploying a central inventory reporting application into a shared AWS account. The application must be able to read items from all the teams' DynamoDB tables.

Which authentication option will meet these requirements MOST securely?

⊗ Integrate DynamoDB with AWS Secrets Manager in the inventory application account. Configure the application to use the correct secret from Secrets Manager to authenticate and read the DynamoDB table. Schedule secret rotation for every 30 days. Storing secrets directly in Secrets Manager and using them across accounts is less secure than using IAM roles for cross-account access, as it requires managing and rotating secrets. It doesn't leverage the built-in security features of IAM roles and STS.

⊗ In every business account, create an IAM user that has programmatic access. Configure the application to use the correct IAM user access key ID and secret access key to authenticate and read the DynamoDB table. Manually rotate IAM access keys every 30 days.

Managing IAM user access keys directly in the application is less secure and more complex than using IAM roles. It introduces the risk of leaked credentials and requires manual key rotation. IAM roles offer temporary credentials, which are more secure.

✓ In every business account, create an IAM role named BU_ROLE with a policy that gives the role access to the DynamoDB table and a trust policy to trust a specific role in the inventory application account. In the inventory account, create a role named APP_ROLE that allows access to the STS AssumeRole API operation. Configure the application to use APP_ROLE and assume the crossaccount role BU_ROLE to read the DynamoDB table.

This solution leverages IAM roles and STS (Security Token Service) for secure cross-account access. It adheres to the principle of least privilege, allowing the application to assume temporary roles in other accounts only when needed. The BU_ROLE in each business account grants access only to its DynamoDB table, and the trust relationship ensures that only the APP_ROLE in the inventory account can assume the role.

⊗ Integrate DynamoDB with AWS Certificate Manager (ACM). Generate identity certificates to authenticate DynamoDB. Configure the application to use the correct certificate to authenticate and read the DynamoDB table.

DynamoDB does not directly integrate with ACM for authentication. ACM is primarily used for managing SSL/TLS certificates for services like API Gateway and Load Balancers. This option is not a valid approach for securing DynamoDB access.

Overall Explanation:

The most secure option is to use IAM roles and STS for cross-account access. This involves creating an IAM role (BU_ROLE) in each business account with permissions to access the DynamoDB table and a trust policy allowing a specific role (APP_ROLE) in the inventory application account to assume it. In the inventory account, the application uses APP_ROLE to assume BU_ROLE in each business account, allowing secure access to the DynamoDB tables using temporary credentials. This approach adheres to the principle of least privilege and avoids managing long-term credentials directly.

[View Discussions](#)[Ask AI](#)

Question 36

Not Attempted

[Compute](#)

A company runs container applications by using Amazon Elastic Kubernetes Service (Amazon EKS). The company's workload is not consistent throughout the day. The company wants Amazon EKS to scale in and out according to the workload. Which combination of steps will meet these requirements with the LEAST operational overhead?

- ☐ Use an AWS Lambda function to resize the EKS cluster.
Using Lambda to resize the EKS cluster would require custom scripting and adds operational overhead. This is not the most efficient approach.
- ☒ Use the Kubernetes Metrics Server to activate horizontal pod autoscaling.
The Kubernetes Metrics Server provides resource usage data, which is essential for horizontal pod autoscaling. This allows Kubernetes to automatically scale the number of pods based on demand.
- ☒ Use the Kubernetes Cluster Autoscaler to manage the number of nodes in the cluster.
The Kubernetes Cluster Autoscaler automatically adjusts the number of nodes in the EKS cluster based on the resource requests of pods. This ensures that the cluster has enough capacity to run the workload.
- ☐ Use Amazon API Gateway and connect it to Amazon EKS.
Amazon API Gateway is used for managing APIs and is not directly involved in scaling the EKS cluster.
- ☐ Use AWS App Mesh to observe network activity.

AWS App Mesh is a service mesh that provides observability and control over microservices. While useful for monitoring, it does not directly address the scaling requirements.

Overall Explanation:

Combining the Kubernetes Metrics Server (option B) to enable horizontal pod autoscaling and the Kubernetes Cluster Autoscaler (option C) to manage the number of nodes in the cluster will achieve automatic scaling of the EKS cluster and container applications based on workload demand with minimal operational overhead. The Metrics Server allows pods to scale, and the Cluster Autoscaler ensures the underlying infrastructure scales to support those pods.

[View Discussions](#)[Ask AI](#)

Question 37

Not Attempted

Serverless



A company runs a microservice-based serverless web application. The application must be able to retrieve data from multiple Amazon DynamoDB tables. A solutions architect needs to give the application the ability to retrieve the data with no impact on the baseline performance of the application. Which solution will meet these requirements in the MOST operationally efficient way?

☐ AWS AppSync pipeline resolvers

AppSync pipeline resolvers require coding, which impacts operational efficiency.

☐ Amazon CloudFront with Lambda@Edge functions

CloudFront primarily caches web content at the edge and is not directly suited for querying DynamoDB tables for applications.

☐ Edge-optimized Amazon API Gateway with AWS Lambda functions

API Gateway and Lambda functions, while versatile, do not inherently provide a direct or optimized solution for querying multiple DynamoDB tables without impacting performance. They would likely require custom coding for data aggregation.

☒ Amazon Athena Federated Query with a DynamoDB connector

Amazon Athena Federated Query, with a DynamoDB connector, enables querying DynamoDB tables using SQL with minimal operational overhead. It allows direct querying without needing to modify the existing application code significantly and efficiently.

Overall Explanation:

Amazon Athena Federated Query, using the DynamoDB connector, provides a direct, SQL-based query mechanism to access multiple DynamoDB tables. This approach requires minimal coding and is the most operationally efficient since it leverages a managed service designed for this type of data access. AppSync pipeline resolvers require coding.

CloudFront and API Gateway don't directly relate to optimizing DynamoDB queries.

[View Discussions](#)[Ask AI](#)

Question 38

Not Attempted

Security



A company wants to analyze and troubleshoot Access Denied errors and Unauthorized errors that are related to IAM permissions. The company has AWS CloudTrail turned on. Which solution will meet these requirements with the LEAST effort?

- ☐ Use AWS Glue and write custom scripts to query CloudTrail logs for the errors.
AWS Glue requires writing custom scripts which involves more effort. It is generally used for ETL operations, not direct querying of logs for troubleshooting.
- ☐ Use AWS Batch and write custom scripts to query CloudTrail logs for the errors.
AWS Batch is used for running batch computing workloads and also requires writing custom scripts, making it a less suitable option for quickly troubleshooting IAM permission errors.
- ☒ Search CloudTrail logs with Amazon Athena queries to identify the errors.
Amazon Athena allows direct querying of CloudTrail logs using SQL. This offers a quick and easy method to identify specific errors related to IAM permissions without requiring custom scripts. It is a cost-effective and efficient way to analyze CloudTrail data.
- ☐ Search CloudTrail logs with Amazon QuickSight. Create a dashboard to identify the errors.
While Amazon QuickSight can visualize data, it requires a data source. It cannot directly search the CloudTrail logs. It would need to be used in conjunction with Athena or another querying service. Creating a dashboard adds extra effort compared to direct querying.

Overall Explanation:

Amazon Athena is the best option as it allows you to directly query CloudTrail logs using SQL to identify Access Denied and Unauthorized errors. It requires the least amount of

effort compared to writing custom scripts with AWS Glue or AWS Batch, and avoids the need to set up a dashboard with QuickSight for initial troubleshooting. Athena provides a simple and efficient way to analyze the logs.

[View Discussions](#)[Ask AI](#)

Question 39

Not Attempted

Monitoring and Logging



A company wants to add its existing AWS usage cost to its operation cost dashboard. A solutions architect needs to recommend a solution that will give the company programmatic access to its usage cost data. The company must be able to access cost data for the current year and forecast costs for the next 12 months. Which solution will meet these requirements with the LEAST operational overhead?

- ☒ Access usage cost-related data by using the AWS Cost Explorer API with pagination.
The Cost Explorer API provides programmatic access to cost data and supports pagination for efficient data retrieval. It allows access to historical and forecasted cost data, minimizing operational overhead compared to manual report generation or custom data pipelines.
- ☐ Access usage cost-related data by using downloadable AWS Cost Explorer report .csv files.
Downloading .csv files requires manual intervention and lacks the programmatic access needed for an automated dashboard. It also increases operational overhead.
- ☐ Configure AWS Budgets actions to send usage cost data to the company through FTP.
Using AWS Budgets actions with FTP introduces unnecessary complexity and maintenance overhead for transferring cost data. While Budgets can track costs, they are not the ideal tool for exporting data programmatically for dashboard integration.
- ☐ Create AWS Budgets reports for usage cost data. Send the data to the company through SMTP.
Similar to FTP, using AWS Budgets reports and SMTP introduces complexity and operational overhead. SMTP is not well-suited for programmatic access to cost data.

Overall Explanation:

The AWS Cost Explorer API offers programmatic access, forecasting capabilities, and pagination, leading to the least operational overhead. The other options require more

manual intervention or introduce unnecessary complexities. Cost Explorer is designed for dashboard integration with APIs and programmatic access.

[View Discussions](#)[Ask AI](#)

Question 40

Not Attempted

Databases



A solutions architect is reviewing the resilience of an application. The solutions architect notices that a database administrator recently failed over the application's Amazon Aurora PostgreSQL database writer instance as part of a scaling exercise. The failover resulted in 3 minutes of downtime for the application. Which solution will reduce the downtime for scaling exercises with the LEAST operational overhead?

- ☐ Create more Aurora PostgreSQL read replicas in the cluster to handle the load during failover.
While read replicas can improve read performance and availability, they do not mitigate downtime associated with writer instance failover during scaling operations.
- ☐ Set up a secondary Aurora PostgreSQL cluster in the same AWS Region. During failover, update the application to use the secondary cluster's writer endpoint.
Setting up a secondary cluster can provide failover capabilities but requires significant operational overhead for managing and synchronizing data between clusters. It also involves application changes to switch endpoints, which isn't the least overhead option.
- ☐ Create an Amazon ElastiCache for Memcached cluster to handle the load during failover.
ElastiCache is a caching service and is suitable for read-heavy workloads, not for handling write operations when the writer instance is unavailable. This option doesn't address the core problem of writer instance downtime during scaling.
- ☒ Set up an Amazon RDS proxy for the database. Update the application to use the proxy endpoint.
Amazon RDS Proxy automatically routes write requests to the healthy writer instance, minimizing downtime by reducing client recovery time after a failover. This solution requires the least amount of operational overhead.

Overall Explanation:

Amazon RDS Proxy minimizes downtime during failover events such as scaling exercises by automatically routing traffic to the new writer instance. It reduces client recovery time and requires the least operational overhead compared to the other options, which involve manual intervention or the management of a secondary database cluster. Therefore, setting up an Amazon RDS proxy and updating the application to use the proxy endpoint is the correct solution. Read replicas only handle read operations and ElastiCache is a caching service and not a failover solution for the primary write instance.

[View Discussions](#)[Ask AI](#)

Question 41

Not Attempted

[Databases](#)

A company has a regional subscription-based streaming service that runs in a single AWS Region. The architecture consists of web servers and application servers on Amazon EC2 instances in Auto Scaling groups behind Elastic Load Balancers. The architecture includes an Amazon Aurora global database cluster that extends across multiple Availability Zones. The company wants to expand globally and to ensure that its application has minimal downtime. Which solution will provide the MOST fault tolerance?

- ☒ Extend the Auto Scaling groups for the web tier and the application tier to deploy instances in Availability Zones in a second Region. Use an Aurora global database to deploy the database in the primary Region and the second Region. Use Amazon Route 53 health checks with a failover routing policy to the second Region.

Auto Scaling groups cannot span AWS Regions.

- ☒ Deploy the web tier and the application tier to a second Region. Add an Aurora PostgreSQL cross-Region Aurora Replica in the second Region. Use Amazon Route 53 health checks with a failover routing policy to the second Region. Promote the secondary to primary as needed.

The question does not mention Aurora PostgreSQL; it mentions Aurora Global Database.

- ✘ Deploy the web tier and the application tier to a second Region. Create an Aurora PostgreSQL database in the second Region. Use AWS Database Migration Service (AWS DMS) to replicate the primary database to the second Region. Use Amazon Route 53 health checks with a failover routing policy to the second Region.

Using DMS for replication will introduce latency and complexity. It also does not leverage Aurora Global Database.

- ✔ Deploy the web tier and the application tier to a second Region. Use an Amazon Aurora global database to deploy the database in the primary Region and the second Region. Use Amazon Route 53 health checks with a failover routing policy to the second Region. Promote the secondary to primary as needed.

This is the most fault-tolerant option. It creates a replicated web and app tier in another region, leverages Aurora Global Database for database replication and fast failover, and employs Route 53 for DNS-based failover.

Overall Explanation:

The best solution for fault tolerance is to deploy the web and application tiers in a second region, utilize an Aurora global database across both regions, and use Route 53 health checks with a failover routing policy. This ensures that if the primary region fails, traffic is automatically routed to the secondary region with minimal downtime. Promoting the secondary database is important to allow write activity in the second region in the event of a primary region failure. Auto Scaling groups do not scale across regions and options involving PostgreSQL are inconsistent with the original question.

 View Discussions

 Ask AI

Question 42

Not Attempted

Application Integration



A data analytics company wants to migrate its batch processing system to AWS. The company receives thousands of small data files periodically during the day through FTP. An on-premises batch job processes the data files overnight, but the batch job takes hours to finish running. The company wants the AWS solution to process incoming data files as soon as possible with minimal changes to the FTP clients that send the files. The solution must delete the incoming data files after the files have been processed successfully. Processing for each file needs to take 3-8 minutes. Which solution will meet these requirements in the MOST operationally efficient way?

- ⊗ Use an Amazon EC2 instance that runs an FTP server to store incoming files as objects in Amazon S3 Glacier Flexible Retrieval. Configure a job queue in AWS Batch. Use Amazon EventBridge rules to invoke the job to process the objects nightly from S3 Glacier Flexible Retrieval. Delete the objects after the job has processed the objects.

This solution involves managing an EC2 instance for FTP, uses a slower storage tier (Glacier), and processes files nightly which doesn't meet the 'as soon as possible' requirement.

- ⊗ Use an Amazon EC2 instance that runs an FTP server to store incoming files on an Amazon Elastic Block Store (Amazon EBS) volume. Configure a job queue in AWS Batch. Use Amazon EventBridge rules to invoke the job to process the files nightly from the EBS volume. Delete the files after the job has processed the files.

This solution involves managing an EC2 instance for FTP and processes files nightly, which doesn't meet the 'as soon as possible' requirement. It also misses on key features of event driven processing

- ⊗ Use AWS Transfer Family to create an FTP server to store incoming files on an Amazon Elastic Block Store (Amazon EBS) volume. Configure a job queue in AWS Batch. Use an Amazon S3 event notification when each file arrives to invoke the job in AWS Batch. Delete the files after the job has processed the files.

AWS Transfer Family cannot directly store files on EBS. While using S3 event notifications is good, Batch is overkill and EBS is not relevant.

- ✔ Use AWS Transfer Family to create an FTP server to store incoming files in Amazon S3 Standard. Create an AWS Lambda function to process the files and to delete the files after they are processed. Use an S3 event notification to invoke the Lambda function when the files arrive.

This solution uses AWS Transfer Family for easy FTP setup, stores files in S3, and uses Lambda for immediate processing upon file arrival via S3 event notifications. Lambda can handle the 3-8 minute processing time per file and delete the file afterward, making it operationally efficient.

Overall Explanation:

The most operationally efficient solution is to use AWS Transfer Family to receive files via FTP directly into S3. An S3 event notification triggers a Lambda function upon file arrival. The Lambda function processes the file (within the 3-8 minute timeframe) and then deletes it from S3. This approach avoids managing FTP servers on EC2, and provides immediate, event-driven processing. AWS Transfer Family provides a managed FTP service with direct integration to S3. Lambda functions can process each file as soon as it arrives triggered by S3 events. Lambda provides fast processing time per file. Lambda can also delete files after processing succeeds. Options A, B, and C involve more operational overhead of managing

FTP servers and batch jobs. Processing latency would be higher waiting for batch windows. Storing files in Glacier (Option A) adds latency for retrieving files. Batch is generally used for much longer jobs.

[View Discussions](#)[Ask AI](#)

Question 43

Not Attempted

[Databases](#)

A company is migrating its workloads to AWS. The company's databases contain transactional and sensitive data. The company wants to use AWS Cloud solutions to improve database security and reduce operational overhead. Which solution BEST meets these requirements?

- ☐ **Migrate the databases to Amazon EC2. Use an AWS Key Management Service (AWS KMS) AWS managed key for encryption.**

Using EC2 requires managing the underlying infrastructure and database software, increasing operational overhead. While KMS encryption helps with security, it doesn't address the operational concerns.

- ☒ **Migrate the databases to Amazon RDS and configure encryption at rest.**

Amazon RDS is a managed database service that reduces operational overhead. Configuring encryption at rest provides data security.

- ☐ **Migrate the data to Amazon S3. Use Amazon Macie for data security and protection.**

While S3 and Macie provide data security, S3 is not designed for transactional database workloads. Macie is more focused on identifying PII rather than securing a database.

- ☐ **Migrate the database to Amazon RDS. Use Amazon CloudWatch Logs for data security and protection.**

While RDS reduces operational overhead, CloudWatch Logs are primarily for monitoring and auditing and do not provide comprehensive data security for the database itself.

Overall Explanation:

Migrating to Amazon RDS and configuring encryption at rest is the best solution. RDS is a managed database service, which reduces operational overhead compared to managing databases on EC2. Encryption at rest ensures data security. S3 is not suitable for

transactional database workloads, and CloudWatch Logs are for monitoring, not database security.

[View Discussions](#)[Ask AI](#)

Question 44

Not Attempted

Storage



A company needs to store its accounting records in Amazon S3. The records must be immediately accessible for 1 year and then must be archived for an additional 9 years. To meet compliance requirements, no one at the company, including administrative users and root users, can be able to delete the records during the entire 10-year period. The records must be stored with maximum resiliency. Which solution will meet these requirements?

- ☐ Store the records in S3 Glacier for the entire 10-year period. Use an access control policy to deny deletion of the records for a period of 10 years.
S3 Glacier is suitable for long-term archival but may not provide immediate accessibility for the first year as required. Also, access control policies can be bypassed by administrative/root users.
- ☐ Store the records by using S3 Intelligent-Tiering. Use an IAM policy to deny deletion of the records. After 10 years, change the IAM policy to allow deletion.
Intelligent-Tiering may not offer the most cost-effective archival storage option for the extended 9-year period. Changing the IAM policy after 10 years to allow deletion also introduces manual steps and potential for human error. IAM policies can also be bypassed.
- ☒ Use an S3 Lifecycle policy to transition the records from S3 Standard to S3 Glacier Deep Archive after 1 year. Use S3 Object Lock in compliance mode for a period of 10 years.
This is the correct answer. The S3 Lifecycle policy transitions the data to Glacier Deep Archive after 1 year for long-term archival. S3 Object Lock in compliance mode prevents any user from deleting or overwriting objects for the specified retention period. Glacier Deep Archive provides very high durability and low storage cost for long-term archival. Compliance mode ensures no one can override or change the retention settings.
- ☐ Use an S3 Lifecycle policy to transition the records from S3 Standard to S3 One Zone-Infrequent Access (S3 One Zone-IA) after 1 year. Use S3 Object Lock in governance mode for a period of 10 years.
While S3 One Zone-IA can provide cost savings, it doesn't offer the same level of resiliency as S3 Glacier Deep Archive for long-term archival. S3 Object Lock in governance mode allows users with special permissions to bypass the retention policy, meaning it doesn't fully prevent deletion.

Overall Explanation:

The correct solution is to use an S3 Lifecycle policy to transition the records from S3 Standard to S3 Glacier Deep Archive after 1 year and use S3 Object Lock in compliance mode for a period of 10 years. This meets all the requirements: immediate access for 1 year, archived for 9 years, unable to delete for 10 years (including by administrative and root users), and maximum resiliency. Compliance mode is crucial as it prevents even root users from deleting the data during the retention period.

[View Discussions](#)[Ask AI](#)

Question 45

Not Attempted

Networking



A company has an online gaming application that supports both TCP and UDP protocols for multiplayer gaming. The company utilizes Amazon Route 53 to direct application traffic to multiple Network Load Balancers (NLBs) distributed across different AWS Regions. To accommodate anticipated user growth, the company aims to enhance application performance and minimize latency for the online game. Which solution will best meet these requirements?

- ☐ Add an Amazon CloudFront distribution in front of the NLBs. Increase the Cache-Control max-age parameter.

CloudFront is designed for caching content and is not suitable for real-time gaming traffic that relies on TCP and UDP. Increasing the Cache-Control max-age will not reduce latency for this type of application.

- ☐ Replace the NLBs with Application Load Balancers (ALBs). Configure Route 53 to use latency-based routing.

ALBs operate at the HTTP layer and do not support TCP or UDP traffic, which is essential for the gaming application. While latency-based routing in Route 53 can help direct traffic to the closest Region, it doesn't inherently optimize the network path.

- ☒ Add AWS Global Accelerator in front of the NLBs. Configure a Global Accelerator endpoint to use the correct listener ports.

AWS Global Accelerator uses the AWS global network to route traffic to the optimal endpoint based on network conditions, improving performance and reducing latency. It supports both TCP and UDP traffic and integrates well with NLBs, making it the appropriate choice.

- ⊗ Add an Amazon API Gateway endpoint behind the NLBs. Enable API caching. Override method caching for the different stages.

API Gateway is designed for API management and not for handling real-time gaming traffic over TCP and UDP. API caching is not relevant in this scenario.

Overall Explanation:

The best solution is to add AWS Global Accelerator in front of the NLBs. Global Accelerator provides static anycast IP addresses and intelligently routes traffic over the AWS global network, minimizing latency and improving performance for TCP and UDP based applications like online games. It provides a fixed entry point to the application endpoints in the optimal AWS location. Configuring a Global Accelerator endpoint to use the correct listener ports is also essential for correct routing.

 View Discussions

 Ask AI

Question 46

Not Attempted

Serverless



A company needs to integrate with a third-party data feed. The data feed sends a webhook to notify an external service when new data is ready for consumption. A developer wrote an AWS Lambda function to retrieve data when the company receives a webhook callback. The developer must make the Lambda function available for the third party to call. Which solution will meet these requirements with the MOST operational efficiency?

- ✔ Create a function URL for the Lambda function. Provide the Lambda function URL to the third party for the webhook.

Function URLs provide a dedicated HTTP(S) endpoint for direct invocation of a Lambda function, offering simplicity and efficiency for webhook integrations.

- ⊗ Deploy an Application Load Balancer (ALB) in front of the Lambda function. Provide the ALB URL to the third party for the webhook.

While an ALB can distribute traffic, it adds unnecessary complexity for a simple webhook integration compared to a Function URL.

- ⊗ Create an Amazon Simple Notification Service (Amazon SNS) topic. Attach the topic to the Lambda function. Provide the public hostname of the SNS topic to the third party for the webhook.

SNS introduces an unnecessary layer for a direct webhook callback. SNS is best suited for pub/sub scenarios, not direct invocation.

- ⊗ Create an Amazon Simple Queue Service (Amazon SQS) queue. Attach the queue to the Lambda function. Provide the public hostname of the SQS queue to the third party for the webhook.

SQS is used for asynchronous message queuing, not direct synchronous invocation via a webhook. It adds unnecessary complexity.

Overall Explanation:

Creating a Lambda Function URL provides the most operationally efficient solution. It generates a direct HTTP(S) endpoint for the Lambda function, simplifying the integration with the third-party webhook. The other options introduce unnecessary components like an ALB, SNS topic, or SQS queue, increasing complexity without adding significant benefit in this scenario. Function URLs offer a simple, managed, and secure way to expose a Lambda function to external services.

[View Discussions](#)[Ask AI](#)

Question 47

Not Attempted

Networking



A company hosts a workload in an AWS Region. Customers access the workload via an Amazon API Gateway REST API, with Amazon Route 53 as the DNS provider. The company requires individual and secure URLs for all customers, ensuring operational efficiency. Which combination of steps achieves this?

- ✓ Register the required domain in a registrar. Create a wildcard custom domain name in a Route 53 hosted zone and record in the zone that points to the API Gateway endpoint.
This step involves creating a wildcard custom domain in Route 53, enabling mapping of individual and secure URLs for all customers to the API Gateway endpoints, simplifying domain management.
- ⊗ Request a wildcard certificate that matches the domains in AWS Certificate Manager (ACM) in a different Region.

Creating certificates in a different region adds unnecessary complexity. Certificates should reside in the same region as the API Gateway to avoid latency and potential configuration issues.

- ⊗ Create hosted zones for each customer as required in Route 53. Create zone records that point to the API Gateway endpoint.

This approach introduces unnecessary complexity by creating and managing numerous hosted zones. A single wildcard domain is more operationally efficient.

- ✓ Request a wildcard certificate that matches the custom domain name in AWS Certificate Manager (ACM) in the same Region.

Requesting a wildcard certificate in the same region simplifies certificate management and avoids region-related issues. The wildcard certificate covers all subdomains, ensuring secure HTTPS communication.

- ⊗ Create multiple API endpoints for each customer in API Gateway.

Creating multiple API endpoints per customer adds complexity. Using a wildcard domain and certificate avoids managing individual domains/certs per customer.

- ✓ Create a custom domain name in API Gateway for the REST API. Import the certificate from AWS Certificate Manager (ACM).

Creating a custom domain name in API Gateway allows associating the custom domain name with the API Gateway endpoints and importing the certificate from ACM for secure communication. This is a core step for enabling custom domains.

Overall Explanation:

The most operationally efficient solution involves registering a wildcard domain and certificate and associating it with the API Gateway. Step A registers the domain and creates a wildcard record in Route 53. Step D requests a wildcard certificate in ACM (same region). Step F configures the custom domain name in API Gateway and imports the ACM certificate. Options B, C, and E introduce unnecessary complexity through multiple regions, hosted zones or API endpoints.

 View Discussions

 Ask AI

Question 48

Not Attempted

Security



A company stores data in Amazon S3. According to regulations, the data must not contain personally identifiable information (PII). The company recently discovered that S3 buckets have some objects that contain PII. The company needs to automatically detect PII in S3 buckets and to notify the company's security team. Which solution will meet these requirements?

- ✓ Use Amazon Macie. Create an Amazon EventBridge rule to filter the SensitiveData event type from Macie findings and to send an Amazon Simple Notification Service (Amazon SNS) notification to the security team.

This is the correct solution. Amazon Macie is designed for discovering sensitive data, including PII, in S3. EventBridge can filter Macie findings based on the `SensitiveData` event type, and SNS can be used to notify the security team.

- ✗ Use Amazon GuardDuty. Create an Amazon EventBridge rule to filter the CRITICAL event type from GuardDuty findings and to send an Amazon Simple Notification Service (Amazon SNS) notification to the security team.

This is incorrect. Amazon GuardDuty is a threat detection service, not a data discovery service. It is not designed to specifically detect PII.

- ✗ Use Amazon Macie. Create an Amazon EventBridge rule to filter the SensitiveData:S3Object/Personal event type from Macie findings and to send an Amazon Simple Queue Service (Amazon SQS) notification to the security team.

This is incorrect. While using Amazon Macie and filtering based on `SensitiveData:S3Object/Personal` is relevant, using Amazon SQS is not ideal for notifying a security team*. SQS is a queuing service, not a direct notification service to humans.

- ✗ Use Amazon GuardDuty. Create an Amazon EventBridge rule to filter the CRITICAL event type from GuardDuty findings and to send an Amazon Simple Queue Service (Amazon SQS) notification to the security team.

This is incorrect for the same reasons as Option B and C. GuardDuty is for threat detection, not PII detection, and SQS is not ideal for human notifications.

Overall Explanation:

The correct solution is to use Amazon Macie for PII detection, filter `SensitiveData` events with Amazon EventBridge, and notify the security team using Amazon SNS. Macie is designed for discovering sensitive data like PII. EventBridge acts as a rule-based event router and SNS provides the publish/subscribe mechanism for direct notification to the security team. GuardDuty is focused on threat detection, and SQS is for queuing messages, not for notifying humans directly.

 View Discussions

 Ask AI

A company wants to build a logging solution for its multiple AWS accounts. The company currently stores the logs from all accounts in a centralized account. The company has created an Amazon S3 bucket in the centralized account to store the VPC flow logs and AWS CloudTrail logs. All logs must be highly available for 30 days for frequent analysis, retained for an additional 60 days for backup purposes, and deleted 90 days after creation. Which solution will meet these requirements MOST cost-effectively?

- ⊗ Transition objects to the S3 Standard storage class 30 days after creation. Write an expiration action that directs Amazon S3 to delete objects after 90 days.

This option suggests transitioning to S3 Standard which is already the default, making the transition redundant. Additionally, S3 Standard is not the most cost-effective for data only needed for backup.

- ⊗ Transition objects to the S3 Standard-Infrequent Access (S3 Standard-IA) storage class 30 days after creation. Move all objects to the S3 Glacier Flexible Retrieval storage class after 90 days. Write an expiration action that directs Amazon S3 to delete objects after 90 days.

Moving to Glacier and then deleting after 90 days incurs unnecessary Glacier minimum storage duration costs. The data should be deleted rather than moved to Glacier if its lifetime is defined.

- ✓ Transition objects to the S3 Glacier Flexible Retrieval storage class 30 days after creation. Write an expiration action that directs Amazon S3 to delete objects after 90 days.

This is the most cost-effective option. Logs are transitioned to Glacier after 30 days when frequent access is no longer required, and the expiration action ensures deletion after 90 days.

- ⊗ Transition objects to the S3 One Zone-Infrequent Access (S3 One Zone-IA) storage class 30 days after creation. Move all objects to the S3 Glacier Flexible Retrieval storage class after 90 days. Write an expiration action that directs Amazon S3 to delete objects after 90 days.

Using S3 One Zone-IA might be cheaper, but the availability is lower and not ideal for backup data. Moving to Glacier and then deleting after 90 days is also cost-inefficient.

Overall Explanation:

The most cost-effective solution is to transition objects to S3 Glacier Flexible Retrieval after 30 days and use an expiration action to delete them after 90 days. This leverages the low-cost storage of Glacier for the backup period and avoids unnecessary storage costs associated with other storage classes or unnecessary data movement prior to deletion. S3

Standard is needed for the first 30 days for frequent analysis, by default when the logs are created. Then moving the data to a cheaper storage when it is not accessed often.

[View Discussions](#)[Ask AI](#)

Question 50

Not Attempted

Security



A company is building an Amazon Elastic Kubernetes Service (Amazon EKS) cluster for its workloads. All secrets that are stored in Amazon EKS must be encrypted in the Kubernetes etcd key-value store. Which solution will meet these requirements?

- ☐ Create a new AWS Key Management Service (AWS KMS) key. Use AWS Secrets Manager to manage, rotate, and store all secrets in Amazon EKS.

Using AWS Secrets Manager does not encrypt the etcd store within the EKS cluster.

- ☒ Create a new AWS Key Management Service (AWS KMS) key. Enable Amazon EKS KMS secrets encryption on the Amazon EKS cluster.

This is the correct solution. Creating a KMS key and enabling EKS KMS secrets encryption encrypts the secrets in the Kubernetes etcd store.

- ☐ Create the Amazon EKS cluster with default options. Use the Amazon Elastic Block Store (Amazon EBS) Container Storage Interface (CSI) driver as an add-on.

The EBS CSI driver is for managing EBS volumes and does not address the encryption of secrets within the etcd store.

- ☐ Create a new AWS Key Management Service (AWS KMS) key with the alias/aws/ebs alias. Enable default Amazon Elastic Block Store (Amazon EBS) volume encryption for the account.

Enabling default EBS volume encryption does not encrypt secrets stored in the Kubernetes etcd key-value store. It encrypts the EBS volumes themselves.

Overall Explanation:

The correct solution is to create a new AWS KMS key and enable Amazon EKS KMS secrets encryption on the Amazon EKS cluster. This directly addresses the requirement of encrypting secrets within the Kubernetes etcd key-value store. Using AWS Secrets Manager or EBS encryption does not fulfill this specific requirement.

[View Discussions](#)[Ask AI](#)

Question 51

Not Attempted

Databases



A company needs to provide its data scientists with near real-time, read-only access to the company's production Amazon RDS for PostgreSQL database. Currently, the database is a Single-AZ deployment. The data scientists will be running complex queries that should not impact the production database's performance. The company requires a solution that provides high availability and is cost-effective. Which solution meets these requirements MOST cost-effectively?

- ☐ Scale the existing production database in a maintenance window to provide enough power for the data scientists.

Scaling the existing database might handle the query load, but it doesn't provide high availability or isolate the data scientists' queries from the production workload.

- ☐ Change the setup from a Single-AZ to a Multi-AZ instance deployment with a larger secondary standby instance. Provide the data scientists access to the secondary instance.

A Multi-AZ instance deployment provides high availability. However, you cannot directly access the secondary standby instance for read operations in a Multi-AZ instance deployment, so it won't meet the read-only access requirement.

- ☐ Change the setup from a Single-AZ to a Multi-AZ instance deployment. Provide two additional read replicas for the data scientists.

While this provides high availability and read access, adding separate read replicas would likely be more expensive than using readable standby instances within a Multi-AZ cluster.

- ☒ Change the setup from a Single-AZ to a Multi-AZ cluster deployment with two readable standby instances. Provide read endpoints to the data scientists.

A Multi-AZ cluster deployment with readable standby instances offers both high availability and the ability to offload read queries to the standby instances. The read endpoints allow the data scientists to connect without impacting the primary instance.

Overall Explanation:

The MOST cost-effective solution is to use a Multi-AZ cluster deployment with readable standby instances (Option D). This provides high availability, allows near real-time read access via read endpoints, and avoids impacting the production database. Multi-AZ

instance deployments do not permit read access to the secondary instance, and adding separate read replicas (option C) to Multi-AZ *instance* deployment is more costly than using readable standby instances in Multi-AZ *cluster* deployment. Scaling the existing instance (Option A) does not address HA.

[View Discussions](#)[Ask AI](#)

Question 52

Not Attempted

[Compute](#)

A company hosts a three-tier web application on AWS, distributed across three Availability Zones. The architecture includes an Application Load Balancer, EC2 web servers storing user session states, and a MySQL database on an EC2 instance. Anticipating traffic surges, the company needs a solution that provides scalability and high availability across all three AZs. Which of the following meets these requirements?

- ☒

Migrate the MySQL database to Amazon RDS for MySQL with a Multi-AZ DB cluster deployment. Use Amazon ElastiCache for Redis with high availability to store session data and to cache reads. Migrate the web server to an Auto Scaling group that is in three Availability Zones.

This solution provides high availability for the database using RDS Multi-AZ, uses Redis for session management which supports HA and offers flexibility, and ensures the web tier is scalable and highly available across multiple AZs.
- ☐

Migrate the MySQL database to Amazon RDS for MySQL with a Multi-AZ DB cluster deployment. Use Amazon ElastiCache for Memcached with high availability to store session data and to cache reads. Migrate the web server to an Auto Scaling group that is in three Availability Zones.

While RDS Multi-AZ and Auto Scaling are good choices, Memcached is primarily used for caching and does not offer the same level of data persistence and features as Redis for session management.
- ☐

Migrate the MySQL database to Amazon DynamoDB. Use DynamoDB Accelerator (DAX) to cache reads. Store the session data in DynamoDB. Migrate the web server to an Auto Scaling group that is in three Availability Zones.

Although DynamoDB provides scalability and high availability, migrating a MySQL database to DynamoDB requires significant application changes. Session data is typically stored in ElastiCache due to its low latency and in-memory capabilities, rather than DynamoDB.

- ⊗ Migrate the MySQL database to Amazon RDS for MySQL in a single Availability Zone. Use Amazon ElastiCache for Redis with high availability to store session data and to cache reads. Migrate the web server to an Auto Scaling group that is in three Availability Zones.

Using a single Availability Zone for the database does not provide high availability, as a failure in that AZ would cause an outage.

Overall Explanation:

The optimal solution is to migrate the MySQL database to RDS for MySQL with a Multi-AZ deployment for HA. ElastiCache for Redis is preferred over Memcached for storing user session states due to its persistence capabilities and rich data structures, and it supports HA. Finally, migrating the web server to an Auto Scaling group across three Availability Zones ensures scalability and availability.

 View Discussions

 Ask AI

Question 53

Not Attempted

Networking



A global video streaming company uses Amazon CloudFront as a content distribution network (CDN). The company wants to roll out content in a phased manner across multiple countries. The company needs to ensure that viewers who are outside the countries to which the company rolls out content are not able to view the content. Which solution will meet these requirements?

- ✔ Add geographic restrictions to the content in CloudFront by using an allow list. Set up a custom error message.
- CloudFront geographic restrictions can be used to control content distribution at the country level, effectively blocking viewers outside the allowed countries and a custom error message can be displayed.
- ⊗ Set up a new URL for restricted content. Authorize access by using a signed URL and cookies. Set up a custom error message.
- Signed URLs and cookies provide access control, but do not provide geo-restriction capabilities. They are primarily for controlling access based on authentication, not geographic location.
- ⊗ Encrypt the data for the content that the company distributes. Set up a custom error message.

Data encryption protects the content but does not provide geo-restriction capabilities. Encryption alone cannot prevent users from accessing content from unauthorized locations.

- ⊗ Create a new URL for restricted content. Set up a time-restricted access policy for signed URLs.

While signed URLs with time restrictions provide access control, they do not offer geo-restriction capabilities. This solution does not prevent users from unauthorized countries from accessing the content within the allowed time frame.

Overall Explanation:

The most suitable solution is to use CloudFront's geographic restrictions feature with an allow list to specify the countries where the content should be accessible. This directly addresses the requirement to restrict access based on geographic location. Custom error messages enhance the user experience for those in blocked regions.

[View Discussions](#)[Ask AI](#)

Question 54

Not Attempted

Databases



A company's core production application, which utilizes Microsoft SQL Server Standard running on a virtual machine (VM), requires a disaster recovery (DR) solution in the AWS Cloud to enhance its on-premises setup. The application demands a recovery point objective (RPO) of 30 seconds or less and a recovery time objective (RTO) of 60 minutes. The company is keen on minimizing costs for the DR solution. Which of the following options offers the MOST cost-effective solution while meeting these requirements?

- ⊗ Configure a multi-site active/active setup between the on-premises server and AWS by using Microsoft SQL Server Enterprise with Always On availability groups.

This solution is likely to be the most expensive due to the use of Microsoft SQL Server Enterprise and Always On availability groups. It might provide excellent RTO and RPO but isn't cost-effective.

- ⊗ Configure a warm standby Amazon RDS for SQL Server database on AWS. Configure AWS Database Migration Service (AWS DMS) to use change data capture (CDC).

This option offers a good balance between RTO, RPO, and cost. AWS DMS with CDC can replicate changes with minimal latency, meeting the RPO. RDS provides a managed SQL Server instance reducing operational overhead.

- ☒ Use AWS Elastic Disaster Recovery configured to replicate disk changes to AWS as a pilot light.

AWS Elastic Disaster Recovery (DRS) in a pilot light configuration can provide an RPO and RTO close to a warm standby approach while keeping cost low. The databases are always on minimizing RPO. Only apps are turned off but it can meet the 60 minutes RTO.

- ☐ Use third-party backup software to capture backups every night. Store a secondary set of backups in Amazon S3.

Backups every night will not meet the RPO of 30 seconds or less. This solution is unsuitable.

Overall Explanation:

Option C (AWS Elastic Disaster Recovery with pilot light) provides a good RTO and RPO with cost optimization. Elastic Disaster Recovery can offer RPO/RTO targets similar to warm standby but maintain the low cost approach of pilot light. Options A is too expensive.

Option B can be used but is more expensive than option C. Option D does not meet the RPO requirements.

 View Discussions

 Ask AI

Question 55

Not Attempted

Storage



A company runs multiple Windows workloads on AWS. The company's employees use Windows file shares that are hosted on two Amazon EC2 instances. The file shares synchronize data between themselves and maintain duplicate copies. The company wants a highly available and durable storage solution that preserves how users currently access the files. What should a solutions architect do to meet these requirements?

- ☐ Migrate all the data to Amazon S3. Set up IAM authentication for users to access files.
- S3 is object storage and requires significant changes to user access methods. It is not suitable for directly hosting Windows file shares and preserving existing access methods.
- ☐ Set up an Amazon S3 File Gateway. Mount the S3 File Gateway on the existing EC2 instances.

While S3 File Gateway provides file protocol access to S3, it's not ideal for preserving existing Windows file share access and is often used for on-premises integration, not primarily within AWS.

- ✓ Extend the file share environment to Amazon FSx for Windows File Server with a Multi-AZ configuration. Migrate all the data to FSx for Windows File Server.

Amazon FSx for Windows File Server provides a fully managed Windows file server, offering high availability (Multi-AZ) and durability while preserving existing Windows file share access methods.

- ✗ Extend the file share environment to Amazon Elastic File System (Amazon EFS) with a Multi-AZ configuration. Migrate all the data to Amazon EFS.

Amazon EFS is not supported by Windows instances, making it unsuitable for this scenario.

Overall Explanation:

Amazon FSx for Windows File Server is the best solution because it's a fully managed Windows file server that supports Multi-AZ for high availability and durability. It also allows users to access files in the same way they currently do. S3 and EFS are not compatible with Windows environments. S3 file gateway is mainly used for on-prem to AWS integration.

 View Discussions

 Ask AI

Question 56

Not Attempted

Databases



A company has an on-premises server that uses an Oracle database to process and store customer information. The company wants to migrate to AWS to achieve higher availability, improve application performance, and offload reporting from its primary database system in the MOST operationally efficient way. Which solution should they implement?

- ✗ Use AWS Database Migration Service (AWS DMS) to create an Amazon RDS DB instance in multiple AWS Regions. Point the reporting functions toward a separate DB instance from the primary DB instance.

Using DMS for migration is a valid approach, but managing multiple regions and separate instances might not be the most operationally efficient. Also, the lack of a managed read replica makes this less ideal.

- ✗ Use Amazon RDS in a Single-AZ deployment to create an Oracle database. Create a read replica in the same zone as the primary DB instance. Direct the reporting functions to the read replica.

A Single-AZ deployment does not provide high availability. Also, read replica in the same zone would have the same single point of failure.

- ⊗ Use Amazon RDS deployed in a Multi-AZ cluster deployment to create an Oracle database. Direct the reporting functions to use the reader instance in the cluster deployment.

Multi-AZ DB clusters are supported only for the MySQL and PostgreSQL DB engines, not Oracle. While RDS for Oracle does support Multi-AZ *instance* deployments, the question specified *cluster*.

- ✔ Use Amazon RDS deployed in a Multi-AZ instance deployment to create an Oracle database. Direct the reporting functions to the read replica.

RDS Oracle deployed in a Multi-AZ instance deployment provides high availability. Read replicas can be used to offload reporting workloads. This option uses only one AWS service to meet all requirements.

Overall Explanation:

Option D is the most operationally efficient because it uses Amazon RDS with a Multi-AZ instance deployment for high availability. It offloads reporting by directing reporting functions to the reader instance. RDS Oracle Multi-AZ instance deployment fulfils high availability and read replicas fulfil offloading the reporting. The other options have drawbacks like Single-AZ deployments or are not supported by Oracle.

 View Discussions

 Ask AI

Question 57

Not Attempted

Serverless



A company is building a web application on AWS. Client access requests to the website are unpredictable and can be idle for extended periods. Only customers with paid subscriptions can sign in and use the application. Which combination of steps will meet these requirements MOST cost-effectively? (Choose three.)

- ✔ Create an AWS Lambda function to retrieve user information from Amazon DynamoDB. Create an Amazon API Gateway endpoint to accept RESTful APIs. Send the API calls to the Lambda function.

Lambda is cost-effective for applications with unpredictable traffic patterns and long periods of inactivity, as you only pay for actual invocations. API Gateway provides the RESTful API endpoint.

- ⊗ Create an Amazon Elastic Container Service (Amazon ECS) service behind an Application Load Balancer to retrieve user information from Amazon RDS. Create an

Amazon API Gateway endpoint to accept RESTful APIs. Send the API calls to the Lambda function.

ECS would incur costs even when idle, making it less cost-effective than Lambda for this scenario.

- ✔ Create an Amazon Cognito user pool to authenticate users.
Cognito user pools are designed for user authentication and management, handling sign-in and user identity verification.
- ✘ Create an Amazon Cognito identity pool to authenticate users.
Cognito identity pools are for authorization (access control), not authentication. User pools handle authentication.
- ✔ Use AWS Amplify to serve the frontend web content with HTML, CSS, and JS. Use an integrated Amazon CloudFront configuration.
Amplify provides a simple way to deploy and host static web content, integrating well with CloudFront for content delivery.
- ✘ Use Amazon S3 static web hosting with PHP, CSS, and JS. Use Amazon CloudFront to serve the frontend web content.
S3 static web hosting does not support server-side scripting like PHP.

Overall Explanation:

The most cost-effective solution involves using Lambda for the backend to handle unpredictable traffic, Cognito User Pools for authentication, and Amplify with CloudFront for serving the frontend. Lambda's pay-per-invocation model is ideal for idle periods. Cognito User Pools manage user authentication. Amplify simplifies frontend deployment and integrates with CloudFront for efficient content delivery. S3 alone cannot handle dynamic content like PHP.

[View Discussions](#)[Ask AI](#)

Question 58

Not Attempted

Networking



A media company uses an Amazon CloudFront distribution to deliver content over the internet. The company wants only premium customers to have access to the media streams and file content. The company stores all content in an Amazon S3 bucket. The company also delivers

content on demand to customers for a specific purpose, such as movie rentals or music downloads. Which solution will meet these requirements?

- ☐ Generate and provide S3 signed cookies to premium customers.
This option is incorrect because signed cookies should be generated using CloudFront, not S3.
- ☒ Generate and provide CloudFront signed URLs to premium customers.
This option is correct. CloudFront signed URLs allow you to control access to individual files. This is suitable for content on demand, like movie rentals or music downloads.
- ☐ Use origin access control (OAC) to limit the access of non-premium customers.
While OAC restricts access to the S3 bucket to only CloudFront, it does not differentiate access based on the customer.
- ☐ Generate and activate field-level encryption to block non-premium customers.
Field-level encryption is designed to protect sensitive data while in transit. It does not restrict content access based on customer type.

Overall Explanation:

CloudFront signed URLs are the most appropriate solution for restricting access to premium customers when content is delivered on demand. They allow you to generate URLs that are valid for a specific period and only accessible to users with the correct credentials. OAC is useful for restricting access to S3, but it doesn't differentiate between users. Field-level encryption focuses on protecting data in transit, not access control. S3 signed cookies are also incorrect. Signed URLs is the best way to solve this problem.

[View Discussions](#)[Ask AI](#)

Question 59

Not Attempted

[Compute](#)

A company runs Amazon EC2 instances in multiple AWS accounts that are individually billed. The company recently purchased a Savings Plan. Due to changes in business requirements, the company has decommissioned a large number of EC2 instances. The company now wants to leverage its Savings Plan discounts across its other AWS accounts. Which combination of actions will achieve these requirements, following AWS best practices? (Choose two.)

- ✓ From the AWS Account Management Console of the management account, enable discount sharing within the billing preferences section.

Enabling discount sharing from the management account allows the Savings Plan benefits to be distributed across all linked accounts within the organization.

- ✗ From the AWS Account Management Console of the account that purchased the existing Savings Plan, enable discount sharing from the billing preferences section and include all accounts.

This option is incorrect. Sharing must be enabled from the AWS Organization management account.

- ✗ From the AWS Organizations management account, use AWS Resource Access Manager (AWS RAM) to share the Savings Plan with other accounts.

AWS RAM is not used for sharing Savings Plan discounts. Discount sharing is managed through the billing preferences in the management account.

- ✓ Create an organization in AWS Organizations in a new payer account. Invite the other AWS accounts to join the organization from the management account.

Creating a new organization in a dedicated payer account (management account) is a best practice for managing billing and applying Savings Plan discounts centrally.

- ✗ Create an organization in AWS Organizations in the existing AWS account with the existing EC2 instances and Savings Plan. Invite the other AWS accounts to join the organization from the management account.

This is not recommended as the management account should be a dedicated account and not run workloads. It should be reserved for management purposes.

Overall Explanation:

To effectively share Savings Plan discounts across multiple AWS accounts, the best approach involves establishing an AWS Organization with a dedicated payer/management account. First, create a new AWS account and use it as the management account for the new organization. Then, invite existing AWS accounts to join the organization. Finally, enable discount sharing from the billing preferences section of the management account. Using a dedicated management account is a best practice. It helps with centralized governance and billing management.

 View Discussions

 Ask AI

Question 60

Not Attempted

Serverless



A retail company uses a regional Amazon API Gateway API for its public REST APIs. The API Gateway endpoint is a custom domain name that points to an Amazon Route 53 alias record. A solutions architect needs to create a solution that has minimal effects on customers and minimal data loss to release the new version of APIs. Which solution will meet these requirements?

- ☒ Create a canary release deployment stage for API Gateway. Deploy the latest API version. Point an appropriate percentage of traffic to the canary stage. After API verification, promote the canary stage to the production stage.
Using a canary release deployment allows incremental rollout of the new API version to a percentage of traffic. This minimizes impact on customers and potential data loss during the release. Total API traffic is separated at random into a production release and a canary release with a pre-configured ratio. The updated API features are only visible to API traffic through the canary. You can adjust the canary traffic percentage to optimize test coverage or performance.
- ☐ Create a new API Gateway endpoint with a new version of the API in OpenAPI YAML file format. Use the import-to-update operation in merge mode into the API in API Gateway. Deploy the new version of the API to the production stage.
This option does not allow for a gradual rollout and could lead to issues if the new API version has problems.
- ☐ Create a new API Gateway endpoint with a new version of the API in OpenAPI JSON file format. Use the import-to-update operation in overwrite mode into the API in API Gateway. Deploy the new version of the API to the production stage.
This option does not allow for a gradual rollout and overwriting the existing API could lead to data loss and significant customer impact.
- ☐ Create a new API Gateway endpoint with new versions of the API definitions. Create a custom domain name for the new API Gateway API. Point the Route 53 alias record to the new API Gateway API custom domain name.
Switching the Route 53 record abruptly to a new API endpoint doesn't allow for gradual testing and could cause downtime or issues if the new API version has problems. This would have a large effect on customers.

Overall Explanation:

A canary release deployment is the best solution because it allows for a gradual rollout of the new API version to a percentage of traffic. This minimizes impact on customers and

potential data loss during the release. Promoting the canary stage after verification ensures that the new version is stable before being fully deployed.

[View Discussions](#)[Ask AI](#)

Question 61

Not Attempted

Networking



A company hosts its primary website with DNS records managed in Amazon Route 53, pointing to an Application Load Balancer (ALB). They need a solution to redirect users to a backup static error page, hosted in an Amazon S3 bucket, if the primary website becomes unavailable. The solution must minimize changes to the existing infrastructure and avoid unnecessary overhead. Which approach is most suitable?

- ☐ Update the Route 53 records to use a latency routing policy. Add the static error page hosted in an Amazon S3 bucket to the records, directing traffic to the most responsive endpoints.

Latency-based routing aims to direct users to the endpoint with the lowest latency, not necessarily to a failover page when the primary site is unavailable. This approach doesn't guarantee redirection during an outage and may increase complexity.

- ☒ Set up a Route 53 active-passive failover configuration. Direct traffic to the static error page hosted in an Amazon S3 bucket when Route 53 health checks determine that the ALB endpoint is unhealthy.

This solution provides automatic failover. Route 53 health checks monitor the ALB's health, and if it becomes unhealthy, traffic is automatically directed to the S3-hosted static error page. This minimizes changes and infrastructure overhead.

- ☐ Set up a Route 53 active-active configuration with the ALB and an Amazon EC2 instance that hosts the static error page as endpoints. Configure Route 53 to send requests to the instance only if the health checks fail for the ALB.

While an active-active setup with health checks could work, it requires managing an EC2 instance, which adds infrastructure overhead that the question explicitly wants to avoid. An active-passive setup with S3 is more cost-effective and simpler.

- ☐ Update the Route 53 records to use a multivalue answer routing policy. Create a health check. Direct traffic to the website if the health check passes, and direct traffic to the static error page hosted in Amazon S3 if the health check fails.

Multivalue answer routing doesn't guarantee that users will be directed solely to the S3 bucket upon ALB failure. It distributes traffic among healthy endpoints. It doesn't implement a true failover

mechanism and may continue to direct traffic to an unhealthy ALB.

Overall Explanation:

The best solution is to set up a Route 53 active-passive failover configuration. Route 53 health checks monitor the ALB's health. If the ALB becomes unhealthy, Route 53 automatically directs traffic to the static error page hosted in an Amazon S3 bucket. This setup minimizes infrastructure overhead and provides an automatic failover mechanism, fulfilling the requirement of redirecting users to the backup page when the primary website is unavailable. Options A and D do not guarantee failover and option C increases infrastructure overhead.

[View Discussions](#)[Ask AI](#)

Question 62

Not Attempted

Storage



A recent analysis of a company's IT expenses highlights the need to reduce backup costs. The company's chief information officer wants to simplify the on-premises backup infrastructure and reduce costs by eliminating the use of physical backup tapes. The company must preserve the existing investment in the on-premises backup applications and workflows. What should a solutions architect recommend?

- ☒ Set up AWS Storage Gateway to connect with the backup applications using the NFS interface.
Using NFS interface may not be compatible with all existing backup applications and workflows designed for tape backups.
- ☒ Set up an Amazon EFS file system that connects with the backup applications using the NFS interface.
Amazon EFS might not be directly compatible with existing backup applications designed to write to tapes.
- ☒ Set up an Amazon EFS file system that connects with the backup applications using the iSCSI interface.
Amazon EFS might not be directly compatible with existing backup applications designed to write to tapes, requiring significant modifications or a complete overhaul of the backup process.

- ✔ Set up AWS Storage Gateway to connect with the backup applications using the iSCSI-virtual tape library (VTL) interface.

Tape Gateway's VTL interface allows replacing physical tapes with virtual tapes in AWS without altering existing backup workflows. It supports leading backup applications and caches virtual tapes on-premises for low-latency access.

Overall Explanation:

The most suitable solution is to use AWS Storage Gateway with the iSCSI-VTL interface. This approach replaces physical tapes with virtual tapes stored in AWS, minimizing disruption to existing backup applications and workflows. It allows the company to leverage its current investment while reducing costs and simplifying the infrastructure by eliminating physical tapes.

 View Discussions

 Ask AI

Question 63

Not Attempted

Application Integration



A company operates data collection sensors at numerous locations, generating a substantial volume of data that is streamed to the company's central systems. The company aims to develop an AWS-based platform to efficiently ingest and process this high-volume streaming data. The solution must be highly scalable and capable of supporting near real-time data collection. Furthermore, the company needs to store the data in Amazon S3 for subsequent reporting and analytical purposes. Which solution would satisfy these requirements while minimizing operational overhead?

- ✔ Use Amazon Kinesis Data Firehose to deliver streaming data to Amazon S3.
Kinesis Data Firehose is a fully managed service designed for delivering real-time streaming data to destinations like Amazon S3. It offers scalability, minimal operational overhead, and near real-time delivery.
- ✘ Use AWS Glue to deliver streaming data to Amazon S3.
AWS Glue is primarily an ETL (Extract, Transform, Load) service and is not optimized for handling high-volume, near real-time streaming data ingestion.
- ✘ Use AWS Lambda to deliver streaming data and store the data to Amazon S3.
While AWS Lambda can be used for data processing, using it directly for high-volume streaming data ingestion and storage to S3 would involve significant operational overhead for scaling and

managing the Lambda functions.

- ⊗ Use AWS Database Migration Service (AWS DMS) to deliver streaming data to Amazon S3.

AWS DMS is designed for database migration and replication, not for real-time streaming data ingestion from sensors.

Overall Explanation:

Amazon Kinesis Data Firehose is the most appropriate solution as it's specifically designed for ingesting and delivering high-volume streaming data to Amazon S3 in near real-time with minimal operational overhead. It automatically scales and manages the data flow, simplifying the process significantly compared to other options.

 View Discussions

 Ask AI

Question 64

Not Attempted

Security



A company has separate AWS accounts for its finance, data analytics, and development departments. Due to cost optimization and security concerns, the company wants to enforce restrictions on which services each AWS account can utilize. Which solution will meet these requirements with the LEAST operational overhead?

- ⊗ Use AWS Systems Manager templates to control which AWS services each department can use.

AWS Systems Manager templates can help automate tasks, but they are not designed for centrally controlling service usage across multiple accounts with minimal overhead.

- ✔ Create organization units (OUs) for each department in AWS Organizations. Attach service control policies (SCPs) to the OUs.

AWS Organizations with SCPs allows you to centrally manage permissions and restrict service usage across multiple accounts. OUs enable you to group accounts logically, and SCPs provide a single point of control for defining allowed services, resulting in the least operational overhead.

- ⊗ Use AWS CloudFormation to automatically provision only the AWS services that each department can use.

CloudFormation is an infrastructure-as-code service, but it doesn't natively provide centralized control and restriction of service usage across multiple accounts. It requires custom scripting and deployment for each account, leading to higher operational overhead.

- ⊗ Set up a list of products in AWS Service Catalog in the AWS accounts to manage and control the usage of specific AWS services.

AWS Service Catalog allows users to deploy approved services, but it does not directly control which services each account can use. While it can limit what specific products can be used it doesn't necessarily prevent other services from being launched outside the catalog, which doesn't solve the core problem.

Overall Explanation:

AWS Organizations, coupled with Service Control Policies (SCPs), enables centralized control over AWS service usage across multiple accounts by grouping them into Organizational Units (OUs). This approach offers the least operational overhead compared to other options like Systems Manager templates or CloudFormation, which would require more manual configuration and maintenance. Service Catalog can control the product usage within the defined catalog, but doesn't limit the services being used.

 View Discussions

 Ask AI

Question 65

Not Attempted

Networking



A company hosts a multi-tier e-commerce application. The application load balancer and web tier reside in public subnets, while a MySQL database on EC2 instances is in private subnets. The database needs to retrieve product catalog and pricing data from a third-party internet-based provider. A solutions architect must design a secure strategy with minimal operational overhead. What should they do?

- ⊗ Deploy a NAT instance in the VPC. Route all internet-based traffic through the NAT instance.

Using a NAT instance increases operational overhead due to the need for management and scaling.

- ✔ Deploy a NAT gateway in the public subnets. Modify the private subnet route table to direct all internet-bound traffic to the NAT gateway.

A NAT Gateway is a managed service that provides secure and scalable outbound internet access for instances in private subnets with minimal operational overhead.

- ⊗ Configure an internet gateway and attach it to the VPC. Modify the private subnet route table to direct internet-bound traffic to the internet gateway.

Attaching an internet gateway to the VPC and routing traffic directly from the private subnets would expose the database instances directly to the internet, reducing security.

- ⊗ Configure a virtual private gateway and attach it to the VPC. Modify the private subnet route table to direct internet-bound traffic to the virtual private gateway.

A Virtual Private Gateway (VPG) is used for connecting to a VPN or Direct Connect, not for general internet access.

Overall Explanation:

The most suitable solution is to deploy a NAT gateway in the public subnets and configure the private subnet route table to route all outbound internet traffic through the NAT gateway. A NAT Gateway provides a secure and scalable way for instances in private subnets to access the internet without being directly exposed, while minimizing operational overhead because it's a managed service. Using a NAT Instance would require more management. An Internet Gateway is not secure. A Virtual Private Gateway is for connections to VPNs or Direct Connect.

[View Discussions](#)[Ask AI](#)[← Back to Exam](#)[Marketing](#)[Company](#)[Affiliate Program](#)[Udemy Alternative](#)[Unlimited Access](#)[Whizlabs Alternative](#)

[Business Unlimited Access](#)

[TutorialsDojo Alternative](#)

[Support](#)

[Legal](#)

[Contact Us](#)

[Terms of service](#)

team@itcertifications.io

[Privacy policy](#)

[Refund policy](#)

[Shipping policy](#)



© 2026, ITcertifications Technologies. All rights reserved.